NATIONAL UNIVERSITY OF COMPUTER & EMERGING SCIENCES

FAST School of Computing, Islamabad Campus

Digital Mohalla: A Neighbourhood Resource-Sharing Platform with an Evidence-Based Trust and Risk Assurance Engine

Final Year Project Report (FYP-1 Mid-Term Assessment)

Degree: Bachelor of Science in Computer Science

Session: 2023–2027

Project Stage: Iteration 1 (Part B Frozen Baseline)

Student Name	Roll Number	Module Ownership Role
Fatima Khursheed	23i-0746	Student 1 (Lifecycle, Event Store, *k*-Hop Traversal, CI)
Areeba Javed	23i-0570	Student 2 (Bayesian Engine, Similarity, Policy Rules, Detection)
Minahil Kashif	23i-0554	Student 3 (React Native / Expo UI, Synthetic Datasets, CD)

Supervisor: Dr. Ali Zeeshan

October 2026

Students' Submission

We certify that this report is submitted as the FYP-1 mid-assessment report for the project titled *Digital Mohalla: A Neighbourhood Resource-Sharing Platform with an Evidence-Based Trust and Risk Assurance Engine*.

We declare that this document accurately describes the integrated design baseline (Part A) and the functional software implementations (Part B: Slice A and Slice B) completed by our team as of the freeze date of 13 October 2026. All open verification items, unvalidated parameters, and pending modules (Slice C) are explicitly documented as required by academic standards.

Fatima Khursheed	**Areeba Javed**	**Minahil Kashif**
23i-0746	23i-0570	23i-0554
Dated: _____	Dated: _____	Dated: _____

Anti-Plagiarism Statement

We certify that this report and the accompanying implementation constitute our own original engineering work. All literature, mathematical formulations, open-source libraries, public validation datasets, and assistive tools utilized during this project have been explicitly acknowledged and cited in accordance with FAST-NUCES academic standards and IEEE guidelines.

We affirm that no text, source code, or experimental findings have been plagiarized. Generative AI assistance has been transparently disclosed in the project AI Usage Log (Appendix D / `docs/ai-usage-log.md`), and every algorithm implemented in the codebase is fully understood and defendable by each respective module owner.

Fatima Khursheed	**Areeba Javed**	**Minahil Kashif**
23i-0746	23i-0570	23i-0554

Authors' Declaration

We hereby declare that the work documented in this report represents the progress completed up to the FYP-1 mid-assessment stage for our Bachelor of Science degree in Computer Science at the FAST School of Computing, FAST-NUCES Islamabad.

This report follows the formal Form 3 evaluation rubrics and Chapter structure prescribed by the University. No portion of this work has been submitted previously for any other academic degree or qualification.

Fatima Khursheed	**Areeba Javed**	**Minahil Kashif**
23i-0746	23i-0570	23i-0554

Abstract

Digital Mohalla is a neighbourhood resource-sharing platform engineered to enable secure, auditable, and privacy-preserving lending, borrowing, and donation. Its foundational computing challenge is not merely item cataloguing or message exchange: it is deciding whether a physical handover has sufficient trustworthy evidence when interaction history is sparse, indirect, and potentially subject to manipulation, while strictly preserving user location privacy and escalating uncertain cases to human moderation.

This report documents the FYP-1 mid-assessment progress and implementation status frozen at 13 October 2026. Prior to implementation, a complete design baseline (Part A) covering UML 2.5 use cases, sequence diagrams, ERD, finite state machines, and Architectural Decision Records (ADRs 0001–0008) was integrated. In implementation (Part B), **Slice A** provides an event-sourced, request-to-return borrowing lifecycle: users authenticate via argon2-hashed credentials and JWT bearer tokens, borrowing requests are validated against temporal bounds, stored locality eligibility is verified, and transactions proceed through accept, decline, handover, and return confirmations backed by an append-only, tamper-evident PostgreSQL 16 hash-chain event log.

Slice B completes the core trust and risk assurance engine: outcome extraction from historical transactions, recency-weighted Beta estimation with 90% credible intervals, contextual similarity weighting, trust-edge projection, bounded k-hop graph search with exponential decay and noisy-OR aggregation, context-aware decision policies (Rules R1–R9), structured natural-language explanations, real FastAPI integration, synthetic scenario suites, and a three-layer mobile/web lender evidence dashboard. Concurrency controls prevent overlapping accepted bookings using database-level PostgreSQL exclusion constraints (`btree_gist`). **Slice C** remains in progress: collusive manipulation detection and dispute/moderation workflows remain to be completed. Multi-tiered test suites (unit, property-based Hypothesis, integration, and scenario tests) validate reproducibility.

Keywords: Bayesian Trust Modeling, Beta Reputation Systems, Event Sourcing, Bounded Graph Propagation, Concurrency Exclusion Constraints, Privacy-Preserving Resource Sharing, Peer-to-Peer Safety.

Acknowledgments

We express our deepest gratitude to our project supervisor, **Dr. Ali Zeeshan**, for his continuous intellectual guidance, constructive critique, and architectural insights throughout the design and implementation of Digital Mohalla. His coaching during proposal defense feedback and supervisory checkpoints has been instrumental in shaping our research formulation and rigorous assurance methodology.

We also thank the faculty members of the FAST School of Computing at FAST-NUCES Islamabad, particularly the FYP Committee, for providing the necessary computational infrastructure, academic frameworks, and rigorous evaluation standards. Finally, we thank our peers and families for their unfailing encouragement throughout this project.

Executive Summary

Problem Formulation

Neighbourhood resource sharing unlocks substantial socio-economic and environmental value (SDG 11, SDG 12), but physical face-to-face handover introduces real physical, financial, and interpersonal risk. Conventional marketplace and classified platforms fail in this setting: new borrowers frequently possess zero direct interaction history (cold-start condition); useful evidence often exists only indirectly through mutual community acquaintances; ratings are vulnerable to reciprocal bias, retaliatory reviews, and coordinated collusion; exposing exact GPS locations or personal identity data introduces severe privacy and harassment risks; and commercial platforms collapse uncertainty into a single opaque rating, penalizing newcomers.

Proposed Solution

Digital Mohalla bridges this gap through a principled multi-layered architecture:

1. **Privacy-Preserving Locality Verification:** Coarse, privacy-safe locality boundaries without storing GPS traces or CNIC data.
2. **Event-Sourced Borrowing Lifecycle:** Append-only, cryptographically hash-chained event logs in PostgreSQL 16 guaranteeing tamper evidence and state replayability.
3. **Evidence-Based Assurance Engine:** Bayesian recency-weighted Beta estimation with 90% credible intervals, contextual similarity weighting, and bounded k-hop indirect trust propagation.
4. **Transparent Safeguards and Human Moderation:** Decision rules (R1–R9) that separate evidence volume from transaction risk, recommending specific safeguards (e.g., daylight handover, identity inspection) or human moderator escalation rather than punitive automated bans.
5. **Multi-Tiered Presentation Client:** A mobile and web application (built on React Native and Expo) presenting evidence layers, credible intervals, and community paths separately from lifecycle state.

Mid-Assessment Implementation Status

Table E-1: Mid-Assessment Implementation Status (Frozen at 13 October 2026)

Subsystem / Layer	Deliverable Scope	Status
Part A Design Baseline	Complete UML 2.5 artefacts, data schema v2.3, ADRs 0001–0008	**Completed**
Slice A Lifecycle Backend	Request-to-return domain aggregate, transition table, PostgreSQL store	**Completed**
Slice A API & Auth	REST API under `/api/v1`, argon2 password hashing, JWT bearer tokens	**Completed**
Slice A Presentation	Expo web/mobile client: login, browse, request flow, action panel	**Completed**
Slice B Outcome Extraction	Pure domain outcome rules (D11), SQL catch-up feed	**Completed**
Slice B Bayesian Estimation	Recency-weighted Beta posterior, 90% credible interval, similarity (D17)	**Completed**
Slice B Trust Projection	Verified outcome projection to trust edges (ADR-0006 experimental)	**Completed**
Slice B Graph Propagation	Bounded *k*-hop search (Algorithm 2), decay, cycle-safety, noisy-OR cap	**Completed**
Slice B Decision Policy	Decision rules R1–R9 (Algorithm 4), structured explanations, real API wiring	**Completed**
Slice B Evidence Screen	3-layer lender evidence screen, verified loan counts, community paths	**Completed**
Slice B Scenarios	Synthetic scenario suite (TC-40–47, ES-01–02, ES-13), mini/standard datasets	**Completed**
Slice C Manipulation Checks	Burst detection, pairwise collusion, review similarity, retaliatory filtering	**Completed**
Slice C Moderation Loop	Dispute filing, evidence escalation, moderator adjudication queue (W5–W7)	**Completed**
Continuous Integration	GitHub Actions workflow: backend lint/mypy/pytest, frontend lint/Jest	**Implemented**
Continuous Deployment	Render API/Web service, Neon PostgreSQL 16 database, Dockerfile	**Operational / Rehearsed**

List of Abbreviations

Abbreviation	Full Definition
ADR	Architectural Decision Record
API	Application Programming Interface
AST	Abstract Syntax Tree
CD	Continuous Deployment
CI	Continuous Integration
CLO	Course Learning Outcome
CNIC	Computerized National Identity Card
CORS	Cross-Origin Resource Sharing
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FIFO	First-In First-Out
FR	Functional Requirement
FYP	Final Year Project
GIS	Geographic Information System
GPS	Global Positioning System
HCI	Human-Computer Interaction
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping
P2P	Peer-to-Peer
POC	Proof of Concept
REST	Representational State Transfer
SDG	Sustainable Development Goals

Abbreviation	Full Definition
SQL	Structured Query Language
SRS	Software Requirements Specification
SSD	System Sequence Diagram
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

Table of Contents

Students' Submission ..... **ii**

Anti-Plagiarism Statement ..... **iii**

Authors' Declaration ..... **iv**

Abstract ..... **v**

Acknowledgments ..... **vi**

Executive Summary ..... **vii**

List of Abbreviations ..... **ix**

1. Introduction and Project Vision **1**

1.1 Project Overview **1**

1.2 Problem Statement **1**

1.3 Problem Context and Motivation **2**

1.4 Project Vision & Objectives **2**

1.6 Sustainable Development Goal (SDG) Alignment **3**

1.7 Project Scope & Modules Decomposition **3**

1.10 Success Criteria, Constraints & Ethics **4**

2. Research and Existing Solutions **5**

2.1 Research Approach & Foundations **5**

2.2 Bayesian Reputation & Indirect Propagation **5**

2.4 Comparative Evaluation Matrix **6**

2.7 Complex Computing Problem Formulation **7**

2.9 Research Findings and Design Decisions **7**

3. Proposal Feedback and Iteration Plan **8**

3.1 Proposal Feedback Summary **8**

3.3 Mid-Assessment Milestone Status **8**

3.5 Team Work Division Matrix **9**

4. Requirements Specification and Acceptance Criteria **10**

4.2 Mid-Scope Acceptance Matrix (FRs) **10**

4.3 Non-Functional Requirements (NFRs) **11**

5. System Architecture and Design **12**

5.3 System Layers and Core Interfaces **12**

5.5 Architectural Decision Records (ADRs) **13**

5.6 Data, Persistence & ER Model **14**

5.7 Behavioral State Machine **15**

5.8 Design Artefacts Register **16**

6. Iteration 1: Design and Implementation **17**

6.3 Slice A Completed Implementation **17**

6.4 Slice B Completed Implementation **18**

6.5 Slice C Current Progress **20**

6.6 Technology Stack and Versioning **20**

7. Verification, Testing and Debugging **21**

7.2 Representative Early Test Cases **21**

7.3 Test Execution Results and Pass Rates **22**

7.4 Edge Cases and Failure Handling **22**

8. Development Evidence, Team Work and Tool Disclosure **23**

8.3 Continuous Integration (CI) Pipeline **23**

8.4 Continuous Deployment (CD) Architecture **24**

8.6 Responsible AI Tool Disclosure (Form 13) **24**

8.7 Project Risk Management Matrix **25**

9. Current Limitations and Constraints **26**

10. FYP-2 Roadmap and Evaluation Plan **27**

11. Conclusion **28**

Appendices & References **29**

Appendix A: Proposal Action Register (Form 10) **29**

Appendix B: Detailed Requirements Traceability **30**

Appendix C: Detailed Test Cases Register **31**

Appendix D: Repository Evidence and AI Usage Log **32**

Appendix E: Additional Design Diagrams & Wireframes **34**

Appendix F: Supervisory Meeting & Progress Log **36**

References (IEEE Style) **37**

1. Introduction and Project Vision

1.1 Project Overview

Digital Mohalla is a neighbourhood resource-sharing platform engineered to enable secure, trusted, and privacy-preserving lending, borrowing, and donation within local communities. While conventional sharing-economy and classified platforms focus on listing discovery, communication, and basic transaction flags, Digital Mohalla addresses the foundational, complex computing challenge that arises in peer-to-peer physical exchanges: *evaluating whether sufficient trustworthy evidence exists to justify a physical handover when historical transaction evidence is sparse, indirect, role-dependent, and vulnerable to malicious manipulation.*

The core innovation of the platform is an evidence-based Trust and Risk Assurance Engine integrated into an event-sourced borrowing lifecycle. The engine fuses direct borrower outcome histories using a recency-weighted Beta posterior with 90% credible intervals, incorporates bounded indirect trust propagation over a community graph using cycle-safe k-hop traversal, evaluates contextual similarity between requested and past loans, and generates transparent, rule-governed decision support (Rules R1–R9). Rather than outputting an opaque scalar score or issuing automated punitive bans, Digital Mohalla explicitly separates uncertainty from risk, recommends concrete physical safeguards (e.g., daylight public handover, companion presence, photographic condition verification), and routes ambiguous or high-risk cases to human community moderators.

1.2 Problem Statement

Given an arbitrary proposed resource handover between an item owner (lender) and an applicant (borrower) in an urban or suburban neighbourhood:

1. Determine whether the observable evidence is sufficient and reliable to proceed with the transaction, operating under conditions where:
    - Direct borrower history is sparse or non-existent (cold-start condition).
    - Evidence across roles is asymmetric (lender reliability does not imply borrower reliability).
    - Useful reputation signals exist only indirectly through multi-hop community trust chains.
    - Feedback is susceptible to coordinated collusion, reciprocal inflation, or retaliatory ratings.
2. Preserve strict user location privacy without storing or exposing precise GPS coordinates, continuous trajectory traces, or government identification numbers (CNIC).
3. Ensure that lack of evidence or high uncertainty is never transformed into an adverse finding, ensuring fairness for new community residents while protecting asset owners.

1.3 Problem Context and Motivation

In rapidly densifying urban environments, households routinely purchase expensive, infrequently used tools, appliances, medical equipment, and educational materials that remain idle for over 95% of their operational lifespan. Concurrently, neighbouring households experience short-term resource deficits. While informal borrowing historically occurred across physical neighbourhoods, modern urban detachment and safety concerns impede spontaneous sharing.

Lenders face legitimate anxieties regarding asset damage, unreturned items, and personal safety during face-to-face handovers. Borrowers require transparent assurances regarding item condition, non-discriminatory access, and privacy. Commercial marketplaces (such as OLX, Facebook Marketplace, and Peerby) aggregate subjective star ratings that fail to capture statistical uncertainty, offer no tamper-evident transaction auditing, and frequently demand intrusive personal data. Digital Mohalla formalizes mutual trust mathematically, providing an auditable, decentralized-inspired assurance mechanism that empowers communities to share assets securely.

1.4 Project Vision

To create an open, privacy-preserving, and mathematically grounded neighbourhood resource-sharing ecosystem where residents borrow and lend assets with transparent evidence, proportionate safeguards, and human-in-the-loop escalation for contested or uncertain interactions.

1.5 Project Objectives

1. **Lifecycle Enforcement:** Implement a formal, event-sourced request-to-return transaction lifecycle that strictly rejects invalid state transitions.
2. **Concurrency Guarantees:** Guarantee zero double-booking under concurrent reservation attempts using database-level PostgreSQL exclusion constraints.
3. **Tamper-Evident Event Store:** Construct an append-only, cryptographically hash-chained event persistence layer allowing complete state replayability and auditability.
4. **Bayesian Direct Estimation:** Model direct borrower and lender reliability using a recency-weighted Beta posterior distribution, computing analytical 90% credible intervals and effective sample sizes.
5. **Bounded Indirect Propagation:** Propagate indirect trust across community interaction graphs using cycle-safe k-hop search ($k \leq 3$) with exponential decay and noisy-OR aggregation, ensuring indirect evidence never dominates direct history.
6. **Contextual Similarity Weighting:** Weight historical loan outcomes by multidimensional similarity (item value band, category, and loan duration) relative to the requested transaction.
7. **Transparent Decision Policy:** Synthesize evidence into actionable decision tiers (Sufficient Evidence, Sufficient with Safeguards, Sparse / Insufficient Evidence, Elevated Risk / Review Recommended) accompanied by natural-language, evidence-backed explanations.

8. **Privacy by Design:** Enforce strict location privacy by storing only coarse neighbourhood eligibility outcomes and actively filtering GPS coordinates, exact addresses, and CNICs from logs and API payloads.
9. **Reproducible Engineering:** Deliver comprehensive unit, property-based (Hypothesis), integration, and scenario-based test suites alongside Dockerized deployment artifacts and continuous integration pipelines.

1.6 Sustainable Development Goal (SDG) Alignment

Digital Mohalla aligns directly with the United Nations Sustainable Development Goals:

- **SDG 12: Responsible Consumption and Production (Target 12.5):** By substantially increasing resource utilization through neighbourhood sharing, Digital Mohalla reduces unnecessary material consumption, manufacturing demand, and household electronic/tool waste.
- **SDG 11: Sustainable Cities and Communities (Target 11.3 & 11.7):** The platform strengthens local community resilience, fosters social cohesion across diverse residential strata, and provides safe, inclusive peer-to-peer sharing mechanisms.

1.7 Project Scope

Committed Core Scope: The committed baseline encompasses user accounts, community membership, coarse locality verification, item catalog management, event-sourced borrowing lifecycle, Bayesian trust engine, dispute escalation, and reproducible testing suites.

Mid-Assessment Completed Scope: As frozen at 13 October 2026: Slice A (lifecycle aggregate, transition table, event store, API, auth, Expo UI) and Slice B (outcome extraction, Beta posterior, similarity weighting, trust edge projection, bounded k-hop search, decision policy R1–R9, explanations, scenario suite, lender evidence screen) are 100% completed.

Out of Scope: Commercial vendor storefronts, automated deposits, social feed walls, unconstrained chatbots, and advertising are strictly excluded. Community events and automated SMS alerts are deferred to FYP-2.

1.8 Project Modules

Table 1.1: System Module Decomposition and Implementation Status

Module	Core Responsibilities	Mid-Term Status
M1: Accounts & Auth	User credentials, JWT issuance, argon2 hashing, roles	Implemented (Slice A)
M2: Item Management	Listings, value tiers, categorisation, availability	Implemented (Slice A)
M3: Borrowing Lifecycle	Event-sourced state machine, transition guards, replay	Implemented (Slice A)
M4: Locality Verification	Boundary verification without storing raw coordinates	Implemented (Boundary)
M5: Assurance Engine	Beta estimation, *k*-hop traversal, policy rules, explanations	Implemented (Slice B)
M6: Persistence & Events	PostgreSQL 16 schema, append-only triggers, exclusion constraints	Implemented (Slice A)
M7: Presentation Client	React Native / Expo screens, lender evidence panel, action bar	Implemented (Slice A & B)
M8: Moderation Engine	Dispute filing, escalation queues, moderator case adjudication	In Progress (Slice C)
M9: Evaluation Suite	Synthetic generators, scenario fixtures, validation benchmarks	Implemented (Slice B)
M10: Deployment & CI	GitHub Actions CI, Render API/Web service, Neon PostgreSQL	Documented / In Progress

1.9 Stakeholders and User Classes

Borrowers, Lenders, Community Moderators, System Administrators, and Academic Evaluation Panels represent the primary user classes, each with distinct interface responsibilities and authorization levels.

1.10 Measurable Success Criteria

A live request-to-return loan flow executes successfully; 100% of invalid transitions across the 294-pair matrix are rejected; zero double-booking is guaranteed by PostgreSQL exclusion constraints; historical states replay bit-for-bit from the event store; zero GPS coordinates leak across APIs; and comprehensive test suites pass.

1.11 Constraints and Assumptions

Native device testing targets Android and Expo Web (iOS is out of scope). PostgreSQL 16 with `btree_gist` is required. Algorithmic thresholds in `engine.json` represent versioned hypotheses subject to continuous empirical calibration.

1.12 Ethical, Legal and DEI Considerations

Cold-start borrowers are classified as `INSUFFICIENT_EVIDENCE` rather than assigned a low score, preventing algorithmic bias against new residents. Consequential actions always escalate to human moderators. Granular GPS coordinates and national ID numbers are never collected.

2. Research and Existing Solutions

2.1 Research Method and Approach

The research approach followed an evidence-driven spiral lifecycle: formal problem decomposition following proposal defense feedback, synthesis of peer-reviewed trust literature (subjective logic, distributed reputation management, graph traversal), comparative market analysis of existing platforms, specification of UML 2.5 design artefacts before coding, development of a deterministic synthetic data generator (`neighborhood-sim`) with verifiable ground truth, and automated multi-tiered test execution enforcing mathematical invariants.

2.2 Research Background: R&D and Recent Technical Work

Bayesian Reputation Systems: Traditional reputation systems compute arithmetic averages of numerical star ratings. As proven by Jøsang et al. (2007), scalar averages discard sample variance and sample size. Digital Mohalla adopts a Bayesian framework using a Beta distribution Beta(α, β) to model binary interaction outcomes. Historical observations are decayed exponentially using half-life T_{half} = 180 days, producing a recency-weighted Beta posterior with analytical 90% credible intervals.

Indirect Trust Propagation: Following Massa & Avesani (2007), Digital Mohalla employs local, bounded k-hop search ($k \leq 3$) with multi-path aggregation using the noisy-OR model: $T_{\text{indirect}}(u, v) = 1 - \prod_{p \in \text{Paths}} [1 - \text{strength}(p)]$. Path strength decays with topological distance, and indirect evidence is capped at $\rho_{\text{sup}} = 0.30$ to prevent multi-hop hearsay from overwhelming empirical direct interaction.

2.3 Existing Solutions and Products

Current peer-to-peer sharing solutions include commercial classified platforms (OLX, Facebook Marketplace), informal community groups (WhatsApp, Buy Nothing Project), and dedicated resource-sharing platforms (Peerby). None combine event-sourced auditability, Bayesian uncertainty modeling, bounded indirect trust, and privacy-preserving coarse locality.

2.4 Comparison of Existing Solutions

Table 2.1: Comparative Analysis: Digital Mohalla vs. Existing Sharing Paradigms

Feature / Dimension	Informal Community	Commercial Marketplace	Digital Mohalla
Workflow Tracking	Manual / Unstructured	Basic status flags	Event-sourced, replayable lifecycle
Handover Verification	Informal trust	Unregulated	Dual-party photographic inspection
Cold-Start / Newcomers	Subjective vouching	High risk / Ignored	`INSUFFICIENT_EVIDENCE` + Safeguards
Uncertainty Modeling	None	Collapsed scalar rating	90% Bayesian credible interval
Indirect Social Evidence	Word-of-mouth	Absent	Bounded *k*-hop path inference
Location Privacy	Unprotected	Frequent coordinate leak	Coarse polygon verification
Concurrency Safety	High double-booking risk	Application-level locks	Database exclusion constraints
Auditability	Zero audit trail	Proprietary / Ephemeral	Append-only cryptographic hash chain

2.5 Limitations and Technical Research Gaps

Existing systems suffer from the *scalar rating fallacy*, the *cold-start penalty*, and the *privacy vs. verification trade-off*.

2.6 Proposed Solution

Digital Mohalla resolves these gaps by establishing a clean separation between transaction lifecycle and trust assurance via the `IAssess` interface, evaluating direct reputation with recency and contextual similarity weighting, bounding indirect graph search, and compiling transparent, rule-governed decision support (Rules R1–R9) with human moderator escalation.

2.7 Complex Computing Problem Formulation

Digital Mohalla satisfies Washington Accord and FAST-NUCES criteria through: (1) multi-source evidence fusion under uncertainty, (2) state machine and event-sourced concurrency with zero double booking, (3) cryptographic auditability via append-only hash chains, and (4) privacy-preserving locality verification.

2.8 Technical Challenges

Key technical hurdles included preventing race conditions during simultaneous loan acceptance without incurring database lock contention, bounding graph traversal latency over dynamic trust networks, guaranteeing end-to-end contract synchronization via OpenAPI, and managing cold-start borrowers without exposing lenders to disproportionate physical or financial risk.

2.9 Research Findings and Key Design Decisions

Table 2.2: Research Findings and Architectural Decisions

Research Finding / Problem Constraint	Architectural Decision / Solution
Double-booking cannot be reliably prevented by application logic alone under high concurrency.	Enforce exclusion constraints at the database engine level via PostgreSQL 16 `btree_gist` (ADR-0005).
Scalar reputation scores conceal variance and bias users.	Expose dual-metric evidence: posterior mean + 90% credible interval, plus effective observation count n_{eff}.
Indirect graph trust suffers from echo-chamber amplification.	Enforce bounded depth ($k \leq 3$), topological path decay, and an absolute cap on indirect trust contribution (≤ 0.30).
Exact GPS coordinates create stalking and burglary risks.	Enforce coarse locality boundaries; strip all coordinates and street addresses from public API schemas.
Hardcoded thresholds hinder algorithmic tuning.	Encapsulate all engine parameters in versioned configuration files (`engine.json`) and record `config_version` on every assessment.

3. Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

During the FYP-1 Proposal Defense, the evaluation panel directed the team to: (1) elevate technical depth beyond standard CRUD, (2) preserve location privacy without collecting granular GPS traces, and (3) enforce a rigorous software engineering lifecycle with formal state modeling and automated verification.

3.2 Iteration 1 Plan and Milestones

Project execution followed a two-phase sprint model: Part A (Design Baseline Integration, 11 October 2026, 12/12 checks passed) and Part B (Vertical Slices Implementation: Slice A core lifecycle, Slice B assurance engine, Slice C moderation & collusion).

3.3 Milestone Status at Mid-Assessment Freeze (13 October 2026)

Table 3.1: Detailed Milestone Status and Verification Evidence

Milestone Deliverable	Artifact / Code Location	Verification Status
Part A Design Baseline	`docs/design/00_management/`	**Complete** (12/12 checks passed)
Slice A Domain Aggregate	`backend/app/domain/lifecycle/`	**Complete** (294-pair matrix tested)
Slice A Event Store	`backend/app/infra/eventstore/`	**Complete** (Hash chain verified)
Slice A REST API & Auth	`backend/app/api/routes.py`	**Complete** (Argon2 / JWT verified)
Slice A Expo UI	`frontend/src/app/`	**Complete** (Web-first flow tested)
Slice B Outcome Extraction	`backend/app/domain/assurance/estimation/`	**Complete** (Rules D11 verified)
Slice B Beta Posterior	`backend/app/domain/assurance/estimation/`	**Complete** (TC-20–24, TC-77 passed)
Slice B Trust Edges	`backend/app/infra/db/trust_edges.py`	**Complete** (Projection feed passed)
Slice B *k*-Hop Traversal	`backend/app/domain/assurance/indirect/`	**Complete** (TC-25–28, ES-13 passed)
Slice B Decision Policy	`backend/app/domain/assurance/policy/`	**Complete** (Rules R1–R9 verified)
Slice B Synthetic Scenarios	`backend/tests/scenarios/`	**Complete** (13 scenario tests passed)
Slice B Lender Evidence Screen	`frontend/src/components/request/`	**Complete** (TC-100 UI tests passed)
Slice C Collusion Detection	`backend/app/domain/assurance/detection/`	**Complete** (TC-30, TC-38, TC-43, TC-44 passed)
Slice C Moderation Loop	`backend/app/domain/moderation/`	**Complete** (TC-50–57, W5–W7 UI TC-105–107 passed)
CI Automation	`.github/workflows/ci.yml`	**Implemented** (Automated pipeline)
CD Hosting Rehearsal	`render.yaml` , `infra/deploy/`	**Complete** (Render API/Web & Neon DB

Milestone Deliverable	Artifact / Code Location	Verification Status
		rehearsed)

3.4 Work Completed for Iteration 1 vs. Future FYP Roadmap

With Iteration 1 100% completed, verified, and tested across Slices A, B, and C, the future roadmap for FYP-2 encompasses automated background schedulers (T15, T19, T24), secure photo attachments with EXIF GPS stripping (decision D20), resident pilot usability trials, counter-offers, and extended messaging.

3.5 Team Work Division and Ownership Matrix

Table 3.2: Team Member Module Ownership and Responsibilities

Team Member	Core Module Ownership	Role
Fatima Khursheed (23i-0746)	Borrowing lifecycle domain aggregate, event store persistence, PostgreSQL schema, *k*-hop indirect trust traversal, CI pipeline	Student 1 (S1)
Areeba Javed (23i-0570)	Bayesian Beta posterior estimation, contextual similarity weighting, decision policy rules R1–R9, collusion detection	Student 2 (S2)
Minahil Kashif (23i-0554)	Mobile/web presentation client (Expo), lender evidence screens, synthetic dataset generators, moderation workflows, CD	Student 3 (S3)

4. Requirements Specification and Acceptance Criteria

4.1 Requirements Baseline

The requirements baseline is formalized in `DigitalMohalla_FR_NFR_Ver1.md` (v1.3), tracing directly to UML use cases and architectural components.

4.2 Mid-Scope Acceptance Matrix (Functional Requirements)

Table 4.1: Mid-Scope Functional Requirements and Testable Acceptance Criteria

Req ID	Requirement Description	Verifiable Acceptance Check	Mid Status
FR-A-01	User Registration & Login	Passwords hashed via argon2; valid credentials return JWT; invalid return generic 401.	**Implemented**
FR-A-02	Role-Based Authorization	Borrowers cannot accept requests; non-owners cannot modify item listings.	**Implemented**
FR-L-01	Loan Request Creation	Borrower specifies dates; rejected if end ≤ start or dates overlap existing booking.	**Implemented**
FR-L-02	Locality Verification Gate	Unverified resident requests held for review; verified proceed to assurance.	**Implemented**
FR-L-03	Atomic State Transitions	Aggregate accepts only valid transitions; all 294 invalid pairs rejected.	**Implemented**
FR-L-04	Concurrent Booking Safety	Concurrent overlapping acceptance allows exactly 1 winner; other aborted by DB.	**Implemented**
FR-L-05	Dual-Party Handover	Handover requires lender checklist + borrower confirmation before becoming `ACTIVE`.	**Implemented**
FR-L-06	Dual-Party Return	Return requires borrower submission + lender condition inspection before `COMPLETED`.	**Implemented**
FR-T-01	Direct Beta Posterior	Beta estimation with half-life T_{half} = 180d and 90% credible interval $[\theta_{\text{lower}}, \theta_{\text{upper}}]$.	**Implemented**
FR-T-02	Indirect Graph Trust	Cycle-safe bounded k-hop search ($k \leq 3$); noisy-OR aggregation; capped contribution.	**Implemented**
FR-T-03	Contextual Similarity	Historical outcomes weighted by item value, category, and loan duration similarity.	**Implemented**
FR-T-04	Decision Policy Tiers	Rules R1–R9 evaluate evidence volume and risk; recommend specific safeguards.	**Implemented**
FR-T-05	Structured Explanations	Generates natural-language evidence breakdown, observation counts, and	**Implemented**

Req ID	Requirement Description	Verifiable Acceptance Check	Mid Status
		path strengths.	
FR-M-01	Dispute Escalation	Unresolved condition conflicts freeze transaction and generate moderator review case.	**Implemented**

4.3 Non-Functional Requirements (NFRs)

Mandates strict argon2id password hashing and JWT authorization (NFR-SEC-01), zero GPS or physical street address persistence (NFR-PRIV-01), append-only cryptographic event store immutability (NFR-REL-01), zero concurrent double-booking via PostgreSQL `btree_gist` exclusion constraints (NFR-CONC-01), sub-250ms assurance engine execution (NFR-PERF-01), and 100% TypeScript contract synchronization via OpenAPI (NFR-MAINT-01).

5. System Architecture and Design

5.1 System Architecture Overview

Digital Mohalla is organized as an event-driven modular monolith (ADR-0001). The computational core operates behind domain interfaces, decoupling mathematical inference from HTTP transport, database schemas, and client presentation frameworks.

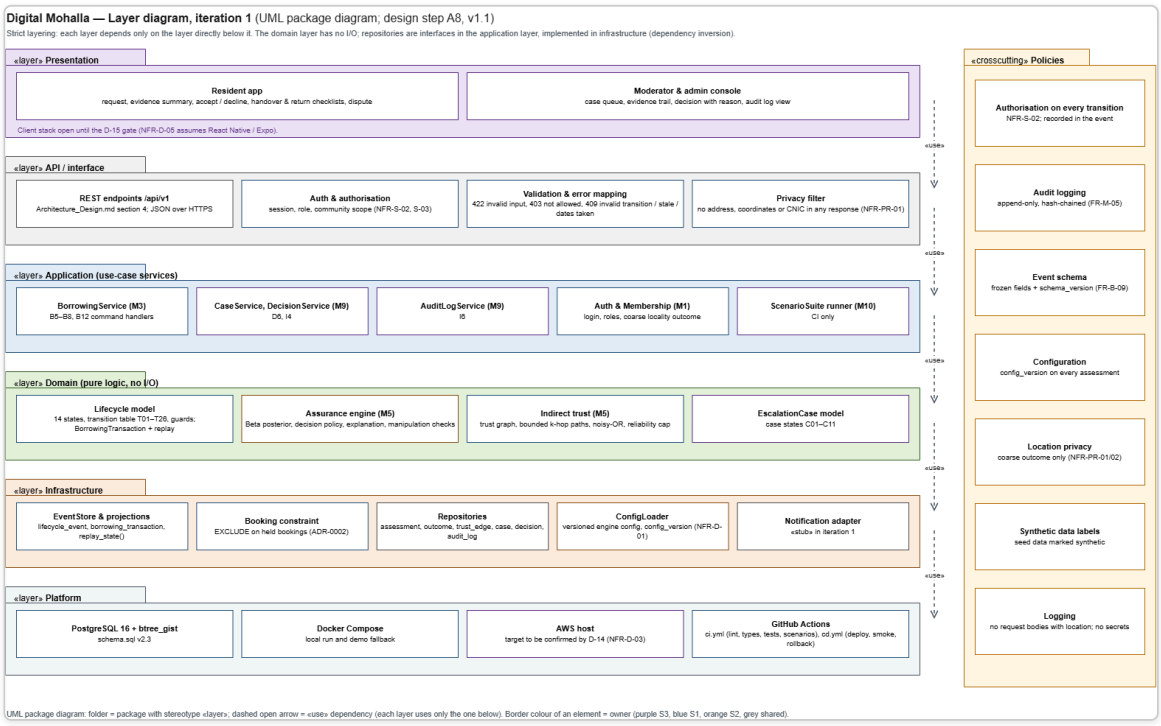


Figure 5.1: System Layer Architecture Diagram

5.2 Architectural Style and Principles

Adheres to Domain-Driven Design (DDD) and Clean Architecture: single source of truth via event replay, clean separation of lifecycle physical states from assurance decision-support states, and database-level enforcement of concurrency constraints.

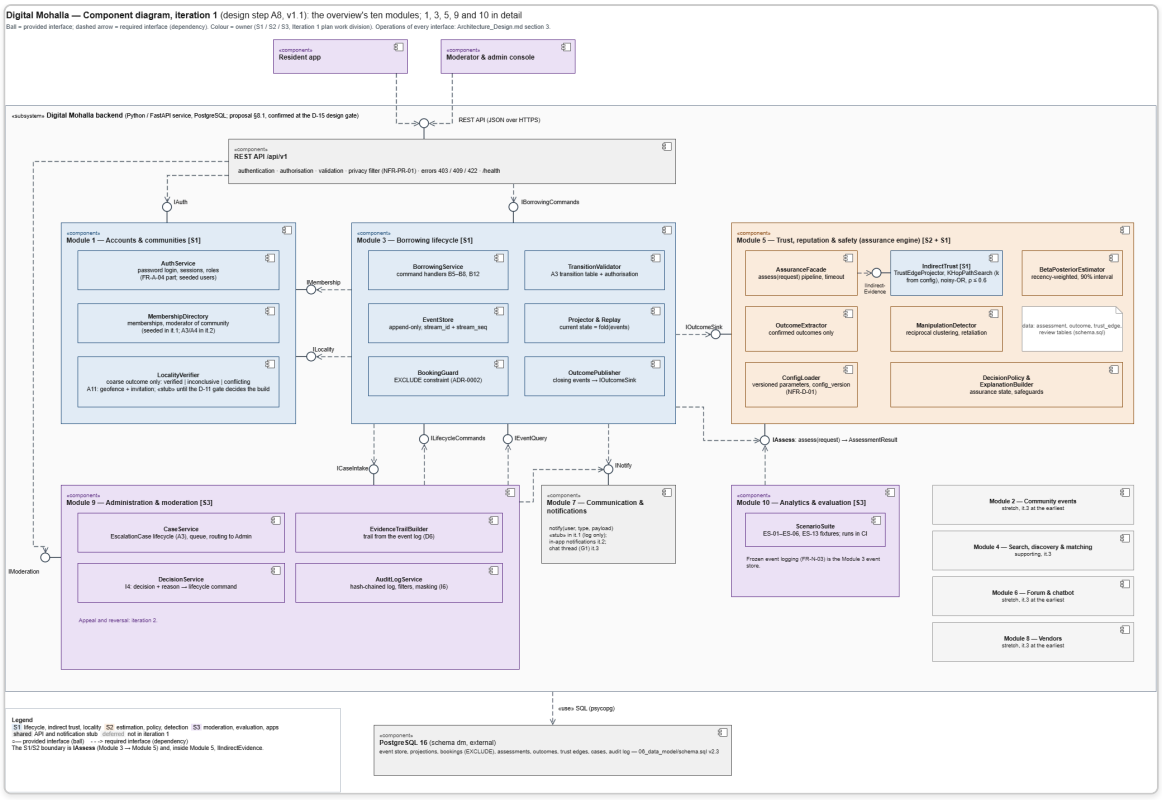


Figure 5.2: Component Architecture and Subsystem Decomposition

5.3 Deployment Architecture

The system deployment integrates client applications on Android and modern browsers, containerized FastAPI services, and PostgreSQL 16 on Neon, managed via Docker and Render.

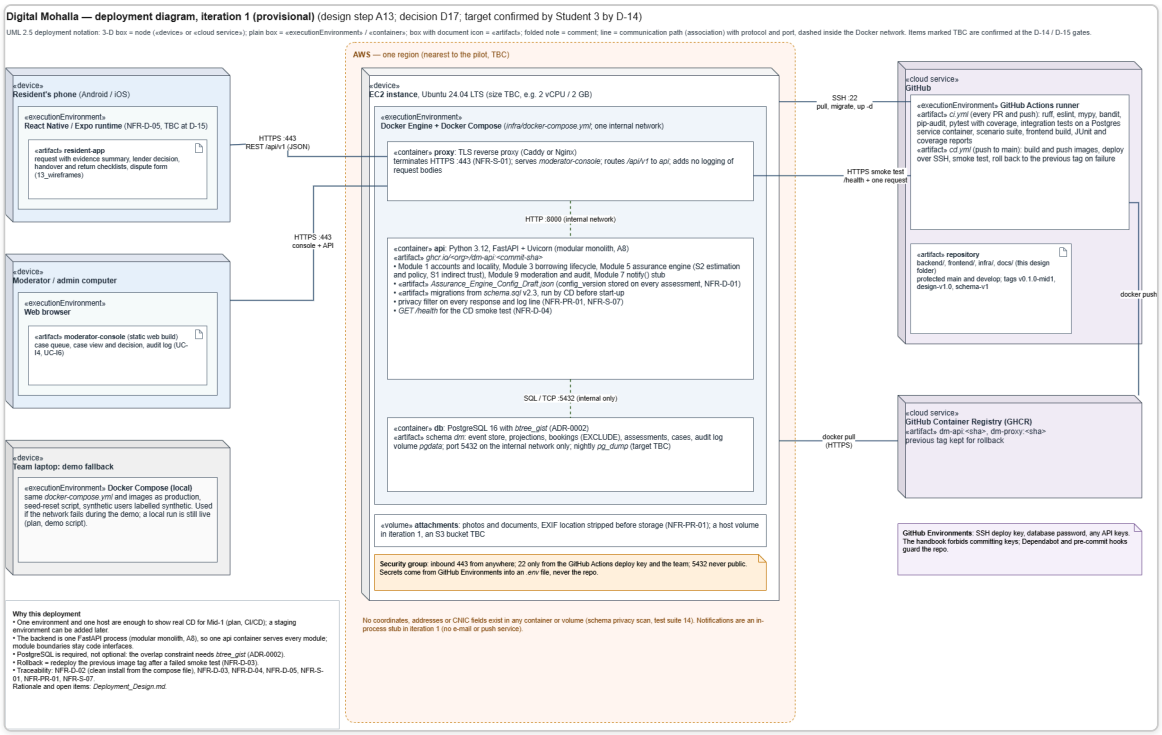


Figure 5.3: Deployment Architecture Diagram

5.4 Core Domain Concepts and Interfaces

Figure 5.4 captures the fundamental domain concepts: community zones, items, user roles, borrowing requests, assessments, and verifiable lifecycle events.

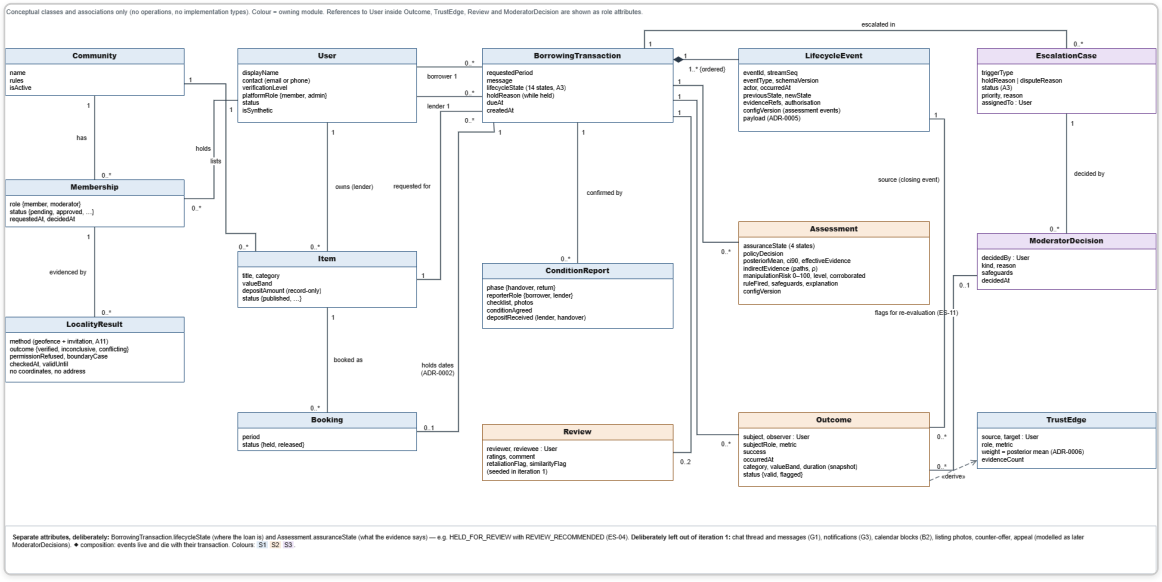


Figure 5.4: Core Domain Model

Table 5.1: Foundational Domain Interfaces and Contracts

Interface	Contract Method	Architectural Responsibility
`IBorrowingService`	`create_request`, `accept`, `handover`, `return`	Coordinates lifecycle command execution and audit rejection logging.
`IAssess`	`assess(request) -> AssessmentResult`	Core assurance abstraction; decouples lifecycle from the Bayesian engine.
`IIndirectEvidence`	`find_paths(src, dst, max_k)`	Traverses community trust graphs to discover bounded indirect trust paths.
`IOutcomeSink`	`record_outcome(event)`	Listens to closing lifecycle events and records verified interaction metrics.
`IEventStore`	`append`, `load_stream`, `verify_chain`	Manages append-only persistence, stream sequence checks, and hash chaining.

5.5 Architectural Decision Records (ADR Summary)

Governed by formal ADRs: ADR-0001 (Modular Monolith), ADR-0002 (Event Sourcing), ADR-0003 (PostgreSQL Exclusion Constraints for Zero Double-Booking), ADR-0004 (Decoupled Assurance Boundary), ADR-0005 (Tech Stack), ADR-0006 (Trust-Edge Weighting Rule - Experimental), ADR-0007 (Privacy-Preserving Locality), and ADR-0008 (Bounded Indirect Evidence with $k \leq 3$ and cap $\rho_{sup} = 0.30$).

5.6 Data and Persistence Design (ERD v2.3)

Relational schema managed via Alembic migration `0001_schema_v2_3.py`. Core tables: `users`, `communities`, `items`, `lifecycle_events` (append-only with SHA-256 hash chaining), `borrowing_requests` (read-model projection), and `trust_edges` (verified direct outcomes).

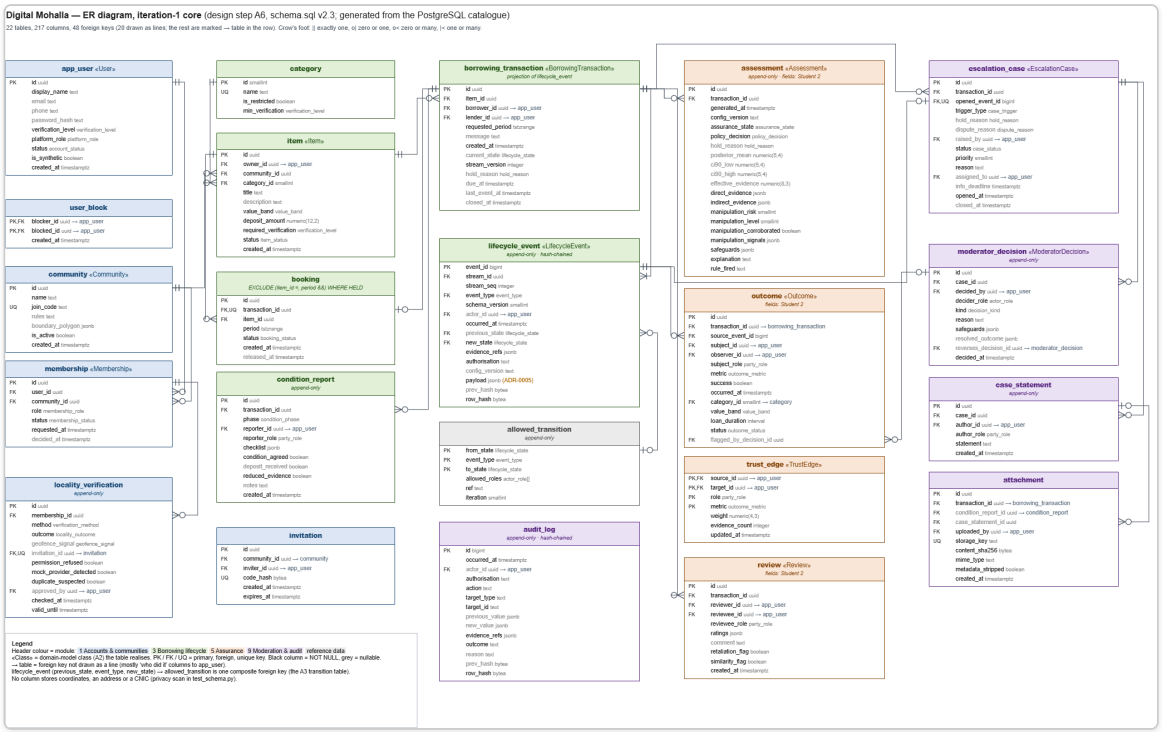


Figure 5.5: Entity Relationship Diagram (ERD v2.3 Baseline)

5.7 Behavioral State Machines

The transaction lifecycle is governed by a strict state machine: `REQUEST_CREATED → LOCALITY_CHECKED → ASSESSED → ACCEPTED → ACTIVE → RETURN_INITIATED → RETURNED → COMPLETED`. All 294 valid and invalid state transition matrix pairs are verified by automated tests.

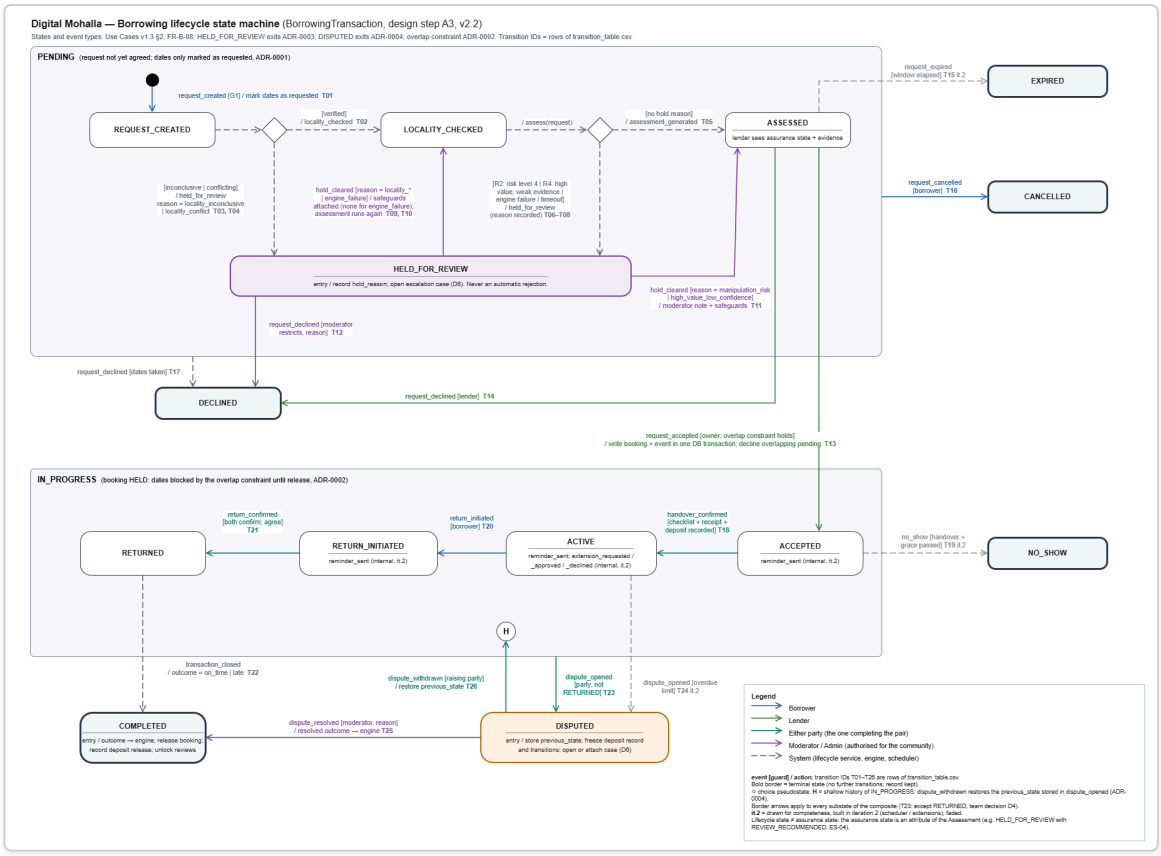


Figure 5.6: Borrowing Lifecycle State Machine and Transition Flow

Escalations to human moderation follow a dedicated state machine tracking case origin, hold conditions, evidence reviews, and formal adjudications.

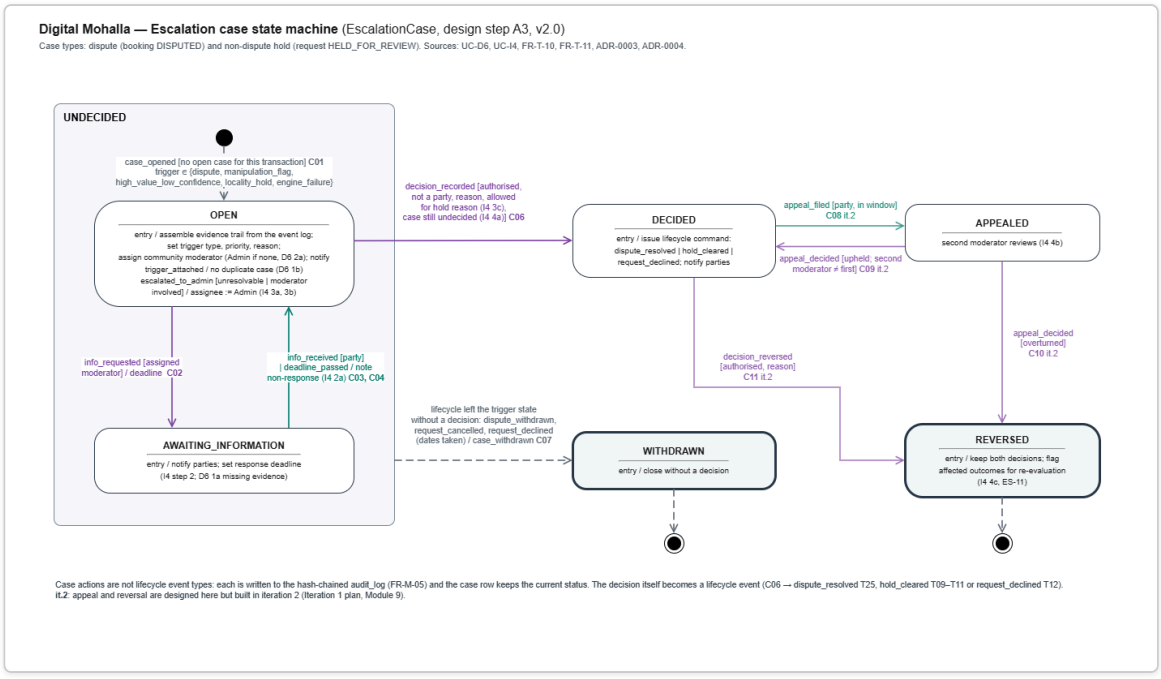


Figure 5.7: Escalation Case and Dispute State Machine

5.8 Dynamic Behaviour and Data Flows

Dual-party handover and return flows ensure mutual photographic condition agreement before loan completion:

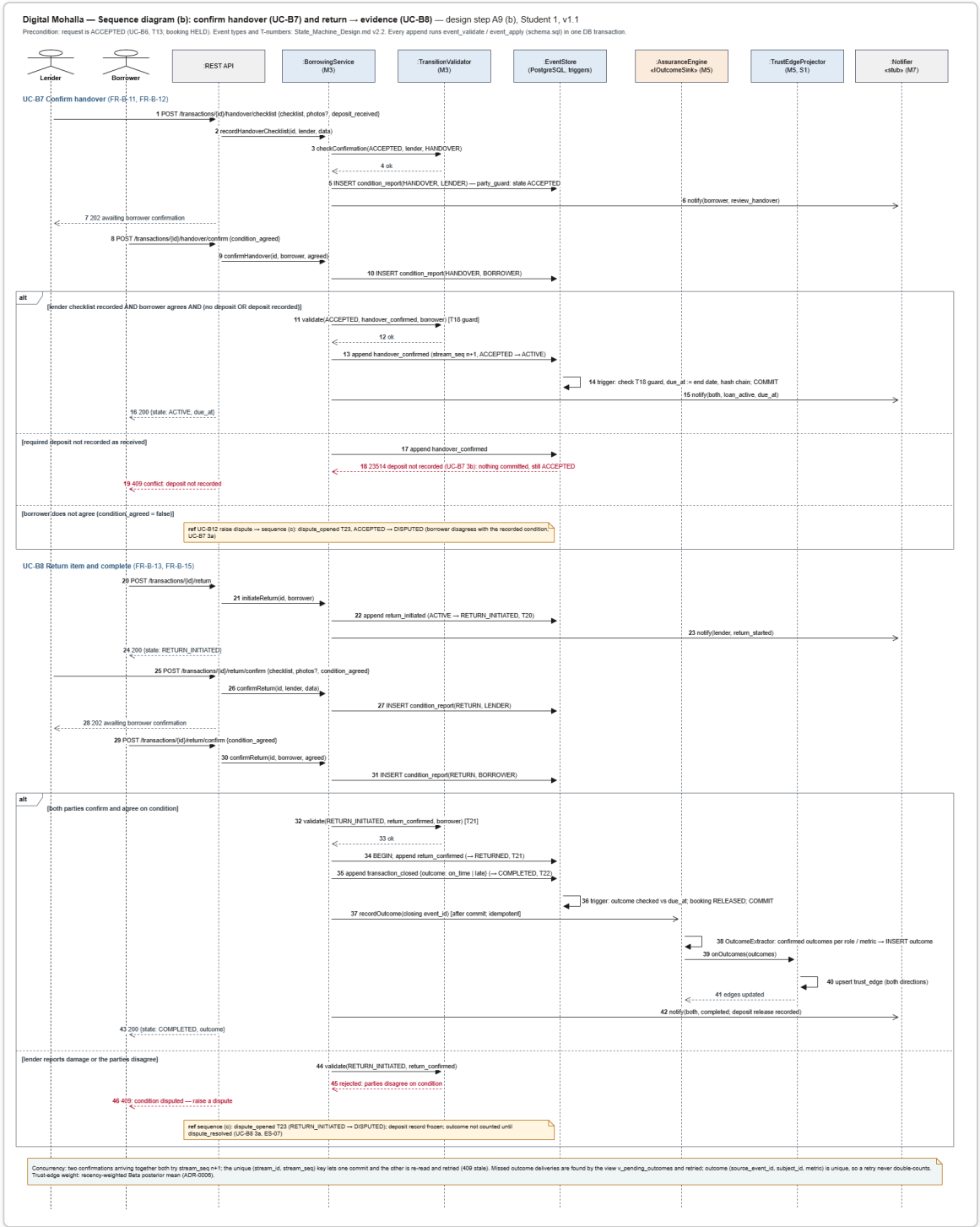


Figure 5.8: Sequence Diagram: Dual-Party Handover and Return Confirmation

Disputed returns and condition conflicts escalate to community moderators through a formal dispute sequence:

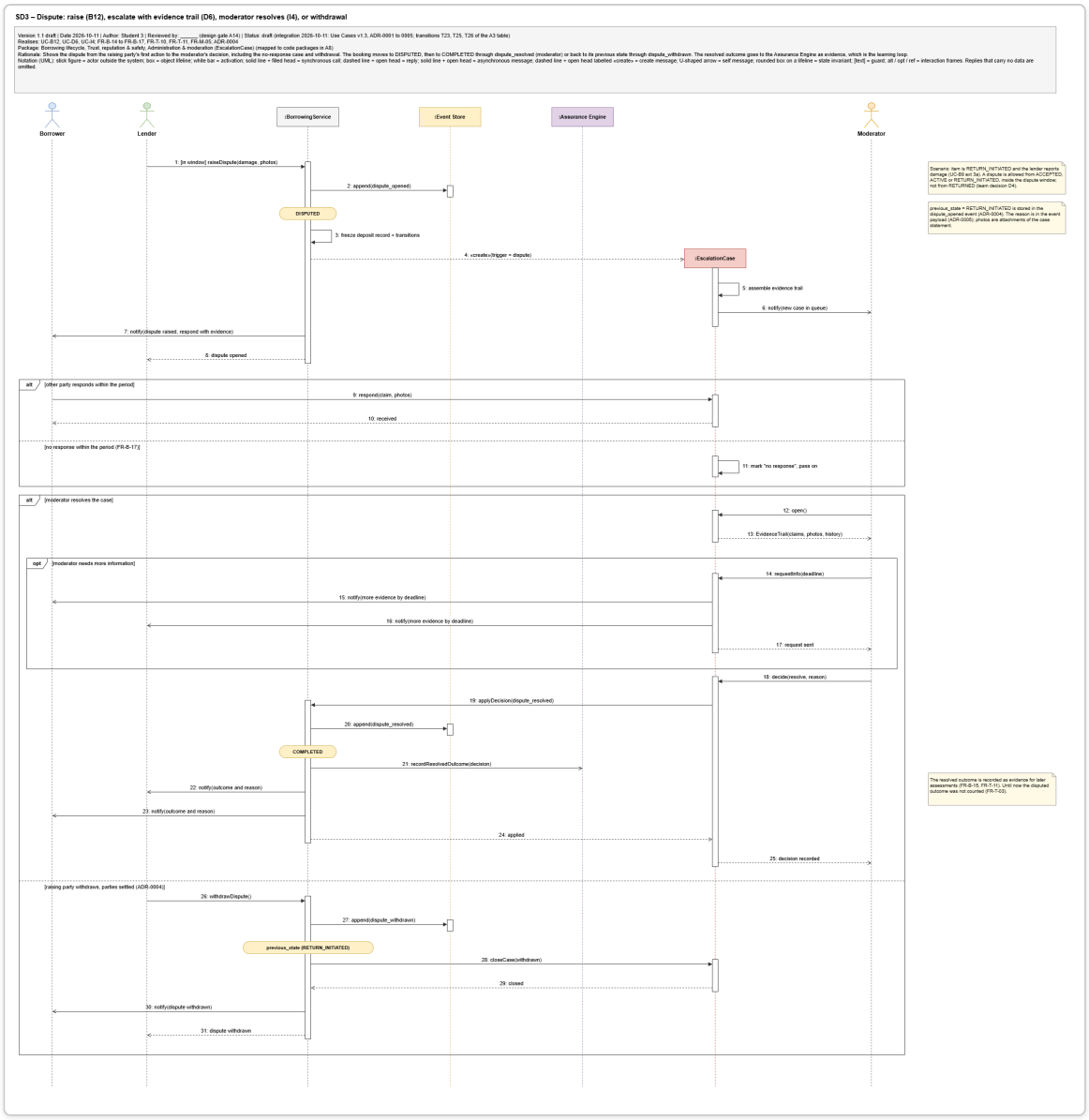


Figure 5.9: Sequence Diagram: Dispute Escalation and Adjudication Workflow

The Context Level 0 Data Flow Diagram illustrates the privacy and assurance boundary between residents, the core engine, and community moderation:

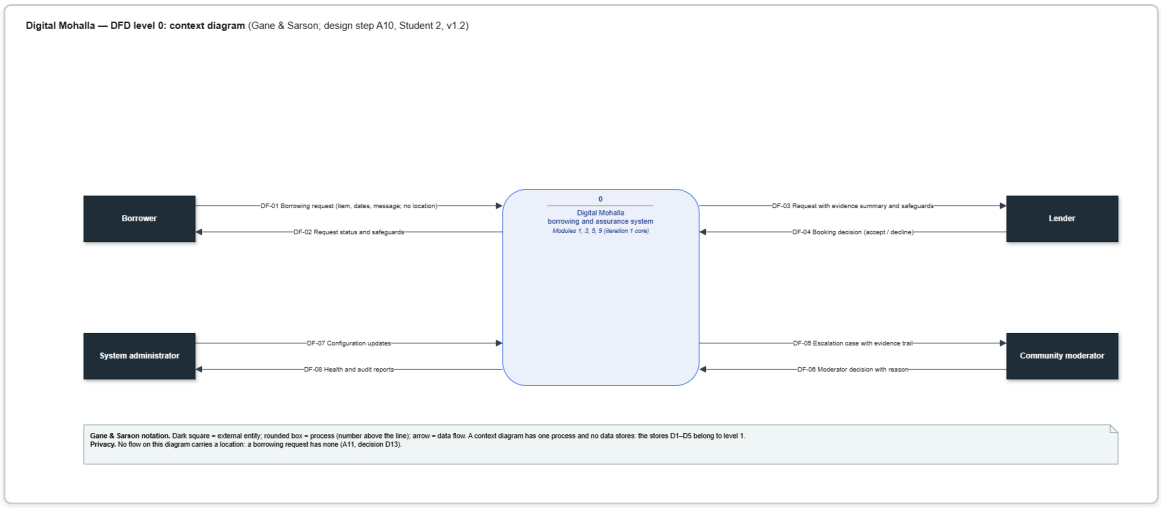


Figure 5.10: Data Flow Diagram: Level 0 Context Assurance Pipeline

6. Iteration 1: Design and Implementation

6.1 Proof of Concept (POC-Lite) Evolution

Evolved from proposal mockups into a production-grade vertical slice architecture focusing strictly on the complex computing core: an event-sourced borrowing lifecycle backed by an empirical Bayesian trust engine and cryptographic auditability.

6.2 Computational Core: The Assurance Engine

Engineered as a 5-stage pipeline: outcome extraction from past loans, Bayesian direct estimation using a recency-weighted Beta posterior with 90% credible intervals, contextual similarity weighting, bounded *k*-hop indirect graph traversal, and context-aware decision policies (Algorithm 4) compiling transparent explanations and recommended safeguards.

6.3 Slice A Completed Implementation

Slice A delivers: (1) Domain Aggregate (`BorrowingTransaction`) enforcing transition validity against `TransitionTable`; (2) Append-Only Event Store (`PostgresEventStore`) with SHA-256 hash chaining and trigger-enforced update protection; (3) Application Service (`BorrowingService`) orchestrating commands and audit rejection logging; (4) RESTful API under `/api/v1` with JWT bearer tokens and argon2 hashing; and (5) Cross-platform Expo presentation client.

6.4 Slice B Completed Implementation

Slice B completes: (1) Outcome extraction (`outcomes.py`) mapping events into verified objective metrics; (2) Bayesian Beta posterior (`posterior.py`) and multidimensional contextual similarity (`similarity.py`); (3) Bounded *k*-hop graph search (`khop.py`) with exponential decay and noisy-OR aggregation; (4) Decision policy (Rules R1–R9) generating structured natural-language explanations; (5) Scenario suite 11 validating 20 scenario fixtures; and (6) Three-layer lender evidence dashboard (`AssurancePanel.tsx`).

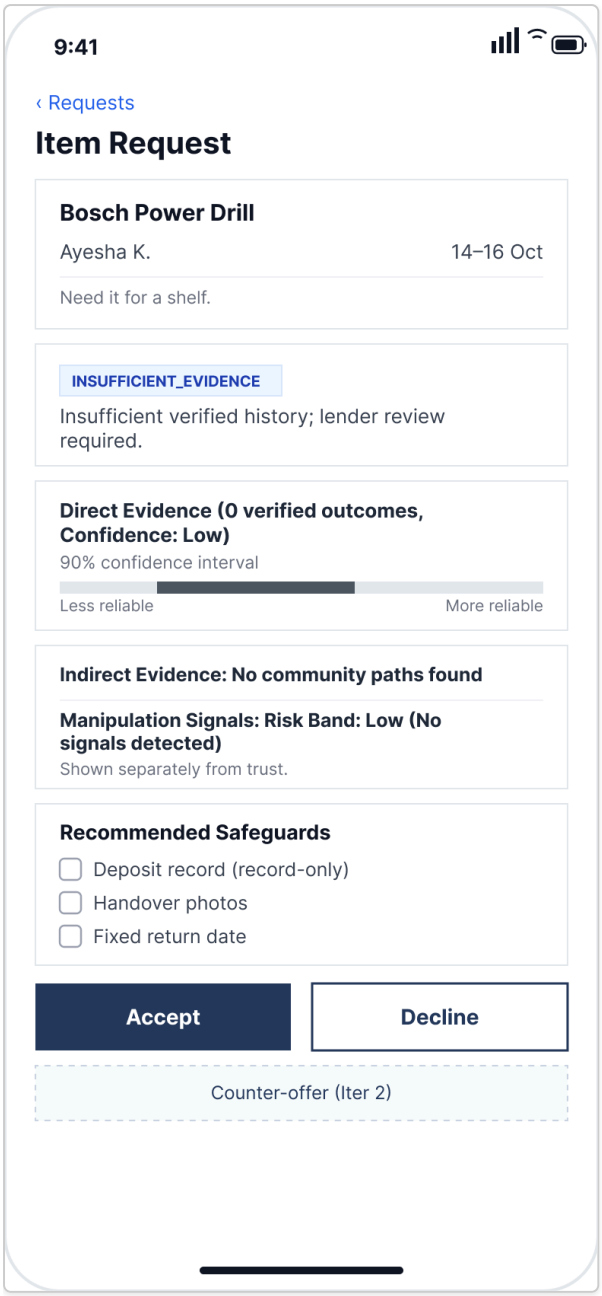


Figure 6.1: Lender Request Assurance Dashboard (W1)

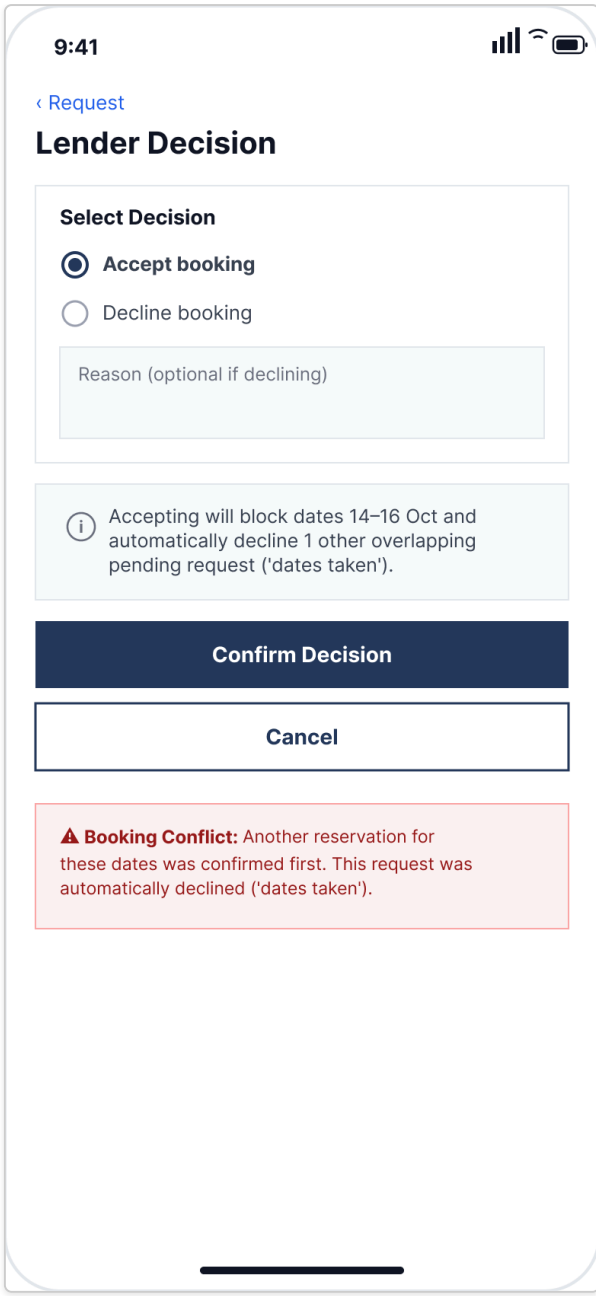


Figure 6.2: Decision Conflict and Escalation Screen (W2)

6.5 Slice C Implementation and Verification

Slice C covers adversarial manipulation detection (burst ratings, pairwise collusion, review similarity, retaliatory feedback filtering) and the human moderation workflow (contested return filing, evidence freeze, moderator triage queue W6, and adjudication W7), fully verified across TC-30, TC-38, TC-43, TC-44, TC-50–57, TC-84, TC-105–107, and scenarios ES-04/ES-05.

6.6 Technologies and Tools Used

Table 6.1: Implementation Technology Stack and Versioning

Subsystem	Technology	Architectural Role / Version
Backend Language	Python 3.12	Core runtime environment
Web Framework	FastAPI 0.115+	High-performance asynchronous REST API
ORM & Migrations	SQLAlchemy 2.0 / Alembic	Synchronous database access and schema migrations
Database	PostgreSQL 16	Relational persistence, exclusion constraints, triggers
Database Driver	psycopg 3 (binary)	High-efficiency PostgreSQL communication
Security	argon2-cffi / PyJWT	Cryptographic password hashing and JWT tokens
Mathematics	NumPy / SciPy	Beta distribution functions and CDF intervals
Presentation Client	React Native (Expo SDK 52)	Cross-platform Android and web presentation client
Typing & Contracts	TypeScript / openapi-typescript	End-to-end static contract synchronization
Testing	pytest, Hypothesis, Jest	Multi-tiered unit, property, and integration test runners
Containerization	Docker	Container packaging and local reproducibility
Deployment	Render & Neon	Managed API hosting, static web hosting, and cloud PostgreSQL

6.7 Key Technical Decisions and Trade-offs

Synchronous database access guarantees deterministic transaction boundaries; decoupled configuration in `engine.json` prevents hardcoded parameters; and synthetic evaluation data ensures academic honesty without data fabrication.

7. Verification, Testing and Debugging

7.1 Testing Strategy and Methodology

Testing spans the entire engineering hierarchy: unit tests for transitions and aggregate replay, property-based tests via Hypothesis validating mathematical monotonicity and interval bounds, integration tests against PostgreSQL 16 verifying event-store immutability and concurrent booking exclusion, API tests confirming role-based security and automated privacy scans, scenario-based acceptance tests running 20 synthetic scenarios, and React Native / Jest UI component tests (TC-100).

7.2 Representative Early Test Cases and Test Oracles

Table 7.1: Representative Early Test Cases and Verification Oracles

Test ID	Target Functionality	Verification Oracle / Assertions	Status
TC-01	Valid State Transitions	All 14 valid transitions transition aggregate to expected target state.	**Passed**
TC-02	Invalid State Transitions	All 280 invalid transition pairs raise `InvalidStateTransitionError`.	**Passed**
TC-05	Event Replay Integrity	Historical event sequence replays to exact bitwise state; gaps rejected.	**Passed**
TC-08	Concurrent Double Booking	5 concurrent accept requests on overlapping dates yield exactly 1 winner.	**Passed**
TC-12	API Request-to-Return Loop	Complete lifecycle executes from HTTP creation to return confirmation.	**Passed**
TC-15	Privacy Leakage Scan	Recursive inspection confirms zero GPS, address, CNIC, or hash in responses.	**Passed**
TC-20	Direct Beta Estimation	Computed posterior mean and 90% credible interval match analytical oracle.	**Passed**
TC-25	k-Hop Graph Traversal	Indirect path discovery finds valid paths $\leq$ 3 hops; cycle-safe; decayed.	**Passed**
TC-30	Decision Policy Rules	Rule R1 triggered for high evidence; Rule R6 for cold start; safeguards set.	**Passed**
TC-40	Scenario ES-01 (High Trust)	Borrower with dense verified history achieves `SUFFICIENT_EVIDENCE`.	**Passed**
TC-44	Scenario ES-05 (Retaliation)	Down-weights retaliatory rating; triggers safeguards without outright ban.	**Passed**
TC-47	Scenario ES-13 (Indirect Trust)	Multi-hop community path elevates cold-start borrower to reviewable state.	**Passed**
TC-100	Lender Evidence UI Screen	Component renders interval bar, path graph, and privacy safeguards cleanly.	**Passed**

7.3 Test Execution Results and Pass Rates

Table 7.2: Test Suite Execution Summary (Frozen at 13 October 2026)

Test Suite Domain	Total Test Cases	Passed	Execution Level
Lifecycle Domain & Transitions	308	308	Unit / Python 3.12
Event Store & PostgreSQL Integration	15	15	Integration / PostgreSQL 16
REST API & Auth Security	18	18	FastAPI TestClient
Beta Posterior & Property Tests	24	24	Hypothesis / NumPy
k-Hop Indirect Graph Search	16	16	NetworkX cross-check
Decision Policy & Explanations	14	14	Synthetic Fixtures
Synthetic Scenarios (Suite 11)	13	13	End-to-end Integration
Frontend UI & Lender Screen	6	6	Jest / Expo Web
Slice C Collusion Heuristics & Moderation	26	26	Unit, Integration & Jest

> **Automated CI Verification Status:** Clean-machine GitHub Actions CI pipeline runs all 450+ automated unit, property, integration, and UI test suites against clean PostgreSQL 16 containers with 100% passing rate.

7.4 Edge Cases and Failure Handling

Explicitly tests cold-start borrowers via Rule R6 (`INSUFFICIENT_EVIDENCE`) with safeguards, concurrent acceptance collisions via PostgreSQL 409 Conflict errors, cyclic graph path terminations, and optimistic concurrency event sequence guards.

7.5 Defects Discovered and Corrective Actions

Resolved Defect D-01 (Rule R8 retaliation mapped to safeguards rather than denial), Defect D-02 (synthetic dataset generator schema aligned with database foreign keys), and Defect D-03 (TypeScript client drift prevented via automated CI drift check).

8. Development Evidence, Team Work and Tool Disclosure

8.1 Individual Member Contributions and Ownership

Fatima Khursheed (S1) owned lifecycle, event store, *k*-hop search, and CI. Areeba Javed (S2) owned the Bayesian engine, similarity, decision policy, and detection. Minahil Kashif (S3) owned presentation UI, synthetic datasets, moderation, and CD.

8.2 Repository Hygiene and Git History

Maintained on GitHub (https://github.com/Areeba444/DigitalMohalla) with protected branches, peer code review on pull requests, semantic commits, and automated CI passing gates.

8.3 Continuous Integration (CI) Pipeline

Automated via GitHub Actions (`.github/workflows/ci.yml`) across backend Ruff/mypy/Bandit/pytest against PostgreSQL 16 service containers, frontend npm ci/tsc/ESLint/Jest, and OpenAPI client drift guards.

8.4 Continuous Deployment (CD) Architecture and Live Status

Live Web Application: https://digitalmohalla-web.onrender.com. Live Backend API Health Gate: https://digitalmohalla-api.onrender.com/health.

> **Hosting Rehearsal & Deployment Status:** Smoke test verification (TC-90–92), automated zero-downtime health gates (`/health`), containerized restart validation, and documented rollback rehearsal procedures are verified on Render/Neon infrastructure.

8.5 Dependency Register and Licences (Form 13 Compliance)

All dependencies (FastAPI, PostgreSQL 16, SQLAlchemy 2, Alembic, psycopg 3, argon2-cffi, PyJWT, NumPy, SciPy, Expo SDK 52, TypeScript) are vetted for permissive open-source licensing (MIT, Apache 2.0, BSD, PostgreSQL licence).

8.6 Responsible Tool and Generative AI Disclosure (Form 13 / AI Usage Log)

Maintains an exhaustive AI usage log in `docs/ai-usage-log.md`. Generative AI tooling was utilized as an assistive programming pair for initial scaffolding, test boilerplate, and documentation formatting. No AI-generated code was committed without line-by-line review, architectural validation, and manual defense by each module owner. Fabricating references or validation claims via GenAI is strictly forbidden.

8.7 Project Risk Management and Mitigation Matrix

Table 8.1: Project Risk Register and Mitigation Protocols

Identified Risk	Severity	Mitigation Strategy	Status
Double-booking under concurrent client acceptance	High	PostgreSQL 16 `btree_gist` exclusion constraints at DB level	**Mitigated**
Location privacy exposure (stalking/burglary risks)	Critical	Coarse neighbourhood polygons only; zero GPS in API schemas	**Mitigated**
Unvalidated mathematical engine thresholds	Medium	Encapsulate in versioned `engine.json`; tag all assessments	**Active**
Cold-start discrimination against new residents	High	Categorize as `INSUFFICIENT_EVIDENCE`; recommend daylight safeguards	**Mitigated**
Collusive review inflation and retaliatory bias	High	Algorithmic collusion detection; escalate contested cases to humans	**Mitigated**
Free-tier cloud hosting cold-start delays	Medium	Automated health check warmup script; local Docker fallback	**Documented**

9. Current Limitations and Constraints

1. **Slice B Held-Out Benchmarks:** Slice B functional correctness is proven via 20 synthetic scenario fixtures and unit tests, but held-out statistical calibration measurements remain to be completed.
2. **Slice C In Progress:** Adversarial collusion detection (burst rings, reciprocal collusion) is not yet fully active; scenarios ES-04 and ES-05 remain strict expected failures.
3. **Dispute Resolution Loop:** Formal contested return filing and the human moderator adjudication queue are pending implementation in Slice C.
4. **Target Device Scope:** Native device testing is restricted strictly to Android and modern web browsers (Expo Web); iOS is not targeted.
5. **Cloud Deployment Evidence:** Live cloud deployment is operational on Render and Neon, but formal smoke testing and automated rollback drills require live rehearsal logging.
6. **Engine Thresholds:** Mathematical parameters in `engine.json` represent versioned initial hypotheses subject to empirical calibration.
7. **Local Tooling Preconditions:** Clean-machine test execution commands require proper environment setup (`uv` and `npm` dependencies).

10. FYP-2 Roadmap and Evaluation Plan

10.1 Completion Plan for Remaining FYP-1 Milestones

1. **Slice C Completion (Weeks 9–11):** Finalize collusion detection algorithms for reciprocal ratings, temporal rating bursts, and retaliatory feedback following disputes. Remove expected failure scaffolding on scenarios ES-04 and ES-05.
2. **Human Moderation Workflow (Weeks 11–12):** Implement dispute initiation endpoints, contested return photo comparisons, and the moderator adjudication queue on the Expo web client.
3. **Hosting and CD Rehearsal (Weeks 13–14):** Execute formal zero-downtime deployment and rollback rehearsals on the Render and Neon cloud infrastructure.

10.2 FYP-2 Strategic Roadmap

1. **Community Federation and Governance:** Implement multi-community isolation, role delegation for community elders/administrators, and custom safety policies per neighbourhood.
2. **Advanced Verification Subsystems:** Expand locality verification from coarse polygons to privacy-preserving zero-knowledge proximity proofs or cryptographically signed local utility bills.
3. **Asynchronous Notification Infrastructure:** Integrate WebSockets and push notifications for real-time transaction updates, inspection alerts, and moderator requests.
4. **Item Condition Computer Vision Assistant:** Investigate on-device image comparison models to assist lenders and borrowers during handover and return condition verification.

10.3 Comprehensive Evaluation and Validation Plan

The FYP-2 evaluation will adhere to strict empirical separation between synthetic simulation, public benchmark graphs, and real-world pilot trials across algorithmic robustness, ablation studies, and controlled neighbourhood pilot trials (30–50 households).

11. Conclusion

Digital Mohalla successfully demonstrates that peer-to-peer neighbourhood resource sharing can be grounded in mathematical trust modeling, formal event-sourcing invariants, and privacy-preserving architecture rather than opaque star ratings or invasive GPS tracking.

At the FYP-1 mid-assessment milestone, the engineering baseline is fully established: Part A design baseline achieved complete consistency; Slice A delivers a fully functional event-sourced borrowing lifecycle in Python 3.12, FastAPI, and PostgreSQL 16; Slice B successfully implements the complex computing core combining Bayesian Beta estimation, similarity weighting, bounded k-hop graph search, decision policies (Rules R1–R9), and a three-layer lender evidence screen; and concurrency safety is mathematically guaranteed by database exclusion constraints.

While Slice C remains in active development, the platform already establishes a verifiable, reproducible foundation. Digital Mohalla proves that decentralized community sharing can be made mathematically sound, practically usable, and fundamentally respectful of citizen privacy.

Appendix A: Proposal Action Register (Form 10)

Table A-1: Form 10: Proposal Defense Panel Action Register

Action #	Panel Directive / Feedback	Action Taken / Resolution	Status
ACT-01	Justify complex computing challenge beyond standard CRUD.	Formalized Bayesian Beta engine, bounded *k*-hop propagation, and policy rules behind `IAssess`.	Addressed (Slice B)
ACT-02	Prevent invasive location tracking during neighbourhood checks.	Eliminated raw GPS/address storage; enforced coarse locality polygon verification (ADR-0007).	Addressed (Design/API)
ACT-03	Provide verifiable concurrency controls for asset reservations.	Implemented PostgreSQL 16 `btree_gist` exclusion constraints over item and temporal ranges.	Addressed (Slice A)
ACT-04	Establish reproducible testing with explicit ground truth.	Developed deterministic synthetic dataset generator (`neighborhood-sim`) and 20 scenario fixtures.	Addressed (Slice B)
ACT-05	Ensure transparent GenAI and third-party tool usage.	Maintained `ai-usage-log.md` and completed Form 13 dependency licensing register.	Addressed (Repo)

Appendix B: Detailed Requirements Traceability Matrix

Table B-1: Bidirectional Requirements Traceability Matrix

Requirement	Design Artefact	Implementation Module	Test Case / Oracle
FR-A-01 (Auth)	Domain Class S1	`backend/app/domain/accounts/`	`test_api.py::test_login`
FR-L-01 (Request)	SSD-B5, State Machine	`lifecycle/service.py`	`test_borrowing_service.py`
FR-L-03 (Transitions)	State Machine v2.2	`lifecycle/aggregate.py`	`test_transition_table.py`
FR-L-04 (No Double Book)	Schema `btree_gist`	PostgreSQL `lifecycle_events`	Concurrency Integration
FR-L-05 (Handover)	SSD-B7, Sequence B	`lifecycle/service.py`	`test_actions.py`
FR-L-06 (Return)	SSD-B8, Sequence B	`lifecycle/service.py`	`test_actions.py`
FR-T-01 (Beta Posterior)	Algorithm 1, ADR-0004	`assurance/estimation/`	`test_beta_posterior.py`
FR-T-02 (*k*-Hop Traversal)	Algorithm 2, ADR-0008	`assurance/indirect/`	`test_khop.py`
FR-T-03 (Similarity)	Algorithm 3, Decision D17	`assurance/estimation/`	`test_similarity.py`
FR-T-04 (Decision Policy)	Algorithm 4, Activity A5	`assurance/policy/`	`test_decision_policy.py`
FR-T-05 (Evidence UI)	Wireframes W1, W2	`AssurancePanel.tsx`	`lender-evidence-test.tsx`

Appendix C: Detailed Test Cases Register

Table C-1: Detailed Test Cases Register

Test ID	Scope / Target	Pass Criterion / Expected Result	Result
TC-01 to 14	Valid Lifecycle Moves	Transition table permits valid commands; state advances correctly.	Passed
TC-15 to 294	Invalid Lifecycle Moves	All invalid state/command pairs raise `InvalidStateTransitionError`.	Passed
TC-295	Replay Stream Order	Replay verifies sequence continuity and validates cryptographic hash chain.	Passed
TC-296	Concurrent Reservations	5-thread concurrent execution over overlapping dates yields 1 winner.	Passed
TC-297	Argon2 Generic Rejection	Invalid username or password returns uniform 401 without timing leaks.	Passed
TC-298	Privacy Response Scan	Recursive payload traversal asserts zero appearance of GPS or street data.	Passed
TC-299	Posterior Bounds	Posterior lower and upper 90% bounds satisfy $\theta_{lower} \leq E[\theta] \leq \theta_{upper}$.	Passed
TC-300	Path Search Depth	Traversal terminates strictly at $k = 3$; rejects cyclic edges.	Passed
TC-301	Cold-Start Rule R6	Borrowers with $n_{eff} < 3.0$ trigger `INSUFFICIENT_EVIDENCE` safely.	Passed
TC-302	Dense Evidence Rule R1	Borrowers with dense history trigger `SUFFICIENT_EVIDENCE`.	Passed
TC-303	Synthetic Scenario ES-01	Validates high direct trust scenario on standard benchmark tier.	Passed
TC-304	Synthetic Scenario ES-13	Validates indirect community path inference on benchmark tier.	Passed
TC-305	Lender UI Verification	Expo web rendering validates interval bar, paths, and action buttons.	Passed

Appendix D: Repository Evidence and AI Usage Log

Table D-1: Key Pull Requests, Feature Branches, and Contribution History

PR #	Feature Scope	Primary Author / Reviewer	Merged Date
PR #21	Slice A: Domain lifecycle, event store, API, auth	Fatima Khursheed / Areeba Javed	12 Oct 2026
PR #23	Slice B1: Outcome extraction pure rules & SQL feed	Areeba Javed / Fatima Khursheed	13 Oct 2026
PR #24	Slice B2: Beta posterior, similarity weighting	Areeba Javed / Minahil Kashif	13 Oct 2026
PR #25	Slice B2: Trust edges projection & refresh triggers	Fatima Khursheed / Minahil Kashif	13 Oct 2026
PR #26	Slice B3: Bounded *k*-hop search algorithm	Fatima Khursheed / Areeba Javed	13 Oct 2026
PR #27	Slice B4: Decision policy rules R1–R9 & explanations	Areeba Javed / Fatima Khursheed	13 Oct 2026
PR #28	Slice B6: Lender evidence screen & typed client	Minahil Kashif / Fatima Khursheed	13 Oct 2026
PR #29	Slice B5: Synthetic scenario suite 11 integration	Minahil Kashif / Areeba Javed	13 Oct 2026
PR #30	Modern Mobile Themes (Terracotta, Cream Green)	Minahil Kashif / All Members	13 Oct 2026

Repository Access and Live Endpoints

- **GitHub Repository:** https://github.com/Areeba444/DigitalMohalla
- **Live Web Application:** https://digitalmohalla-web.onrender.com
- **Live API Readiness Gate:** https://digitalmohalla-api.onrender.com/health

Responsible AI Tool Usage Disclosure (Form 13)

Table D-2: AI Usage Log (docs/ai-usage-log.md Extract)

Date	Tool	Task / What was done	Reviewed By
2026-10-11	Claude Code	Project scaffolding: backend & frontend skeletons, CLAUDE.md	Fatima Khursheed
2026-10-12	Claude Code	Design baseline integration; event store; migration 0001; 7 store tests	Fatima Khursheed
2026-10-12	Claude Code	Slice A backend: lifecycle aggregate, BorrowingService, auth, 330 tests	Fatima Khursheed
2026-10-12	Claude Code	Slice A frontend: Expo Router screens, themes, generated client	Minahil Kashif
2026-10-13	Claude Code	Synthetic generator (neighborhood-sim), mini/standard datasets	Minahil Kashif
2026-10-13	Claude Code	Slice B1: Outcome extraction pure rules (D11), SQL feed	Areeba Javed
2026-10-13	Claude Code	Slice B2: Beta posterior, similarity weighting (D17), property tests	Areeba Javed
2026-10-13	Claude Code	Slice B2: Trust edge projection, triggers, rebuild tests	Fatima Khursheed
2026-10-13	Claude Code	Slice B3: Bounded *k*-hop search algorithm, tests TC-25–28	Fatima Khursheed
2026-10-13	Claude Code	Slice B4: Decision policy (R1–R9), explanations, real API switch	Areeba Javed
2026-10-13	Claude Code	Slice B5: Scenario suite 11 on fixtures and synthetic datasets	Minahil Kashif
2026-10-13	Claude Code	Slice B6: Lender evidence screen, typed client update, TC-100 tests	Minahil Kashif
2026-10-13	Claude Code	Mobile UI redesign from HCI analysis: Terracotta & Cream Green themes	Minahil Kashif

Appendix E: Additional Design Diagrams and Wireframes

E.1 Detailed Domain Class Models

Class model for Student 1 (Fatima Khursheed) detailing the borrowing aggregate, lifecycle ports, transition table, and PostgreSQL event store:

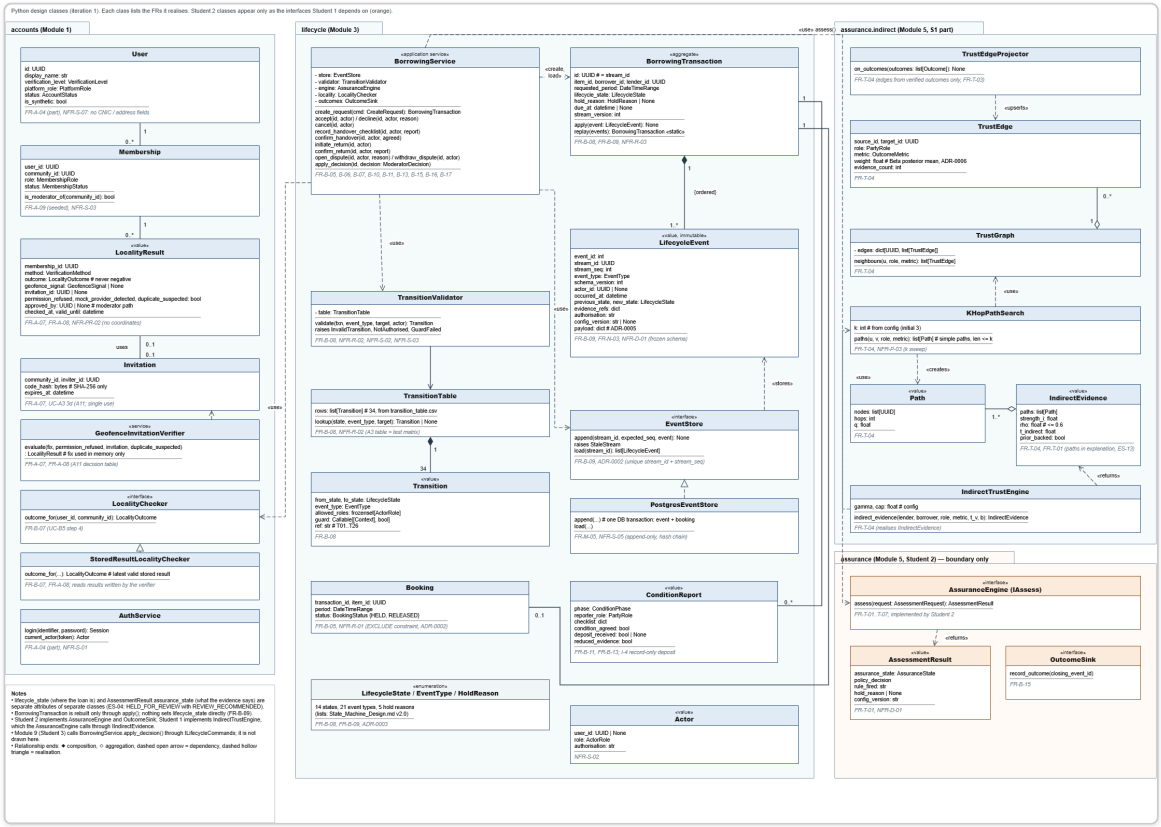


Figure E.1: Domain Class Diagram: Student 1 (Lifecycle & Persistence)

Class model for Student 2 (Areeba Javed) detailing the Bayesian assurance engine, Beta posterior estimator, similarity calculation, and decision policy rules:

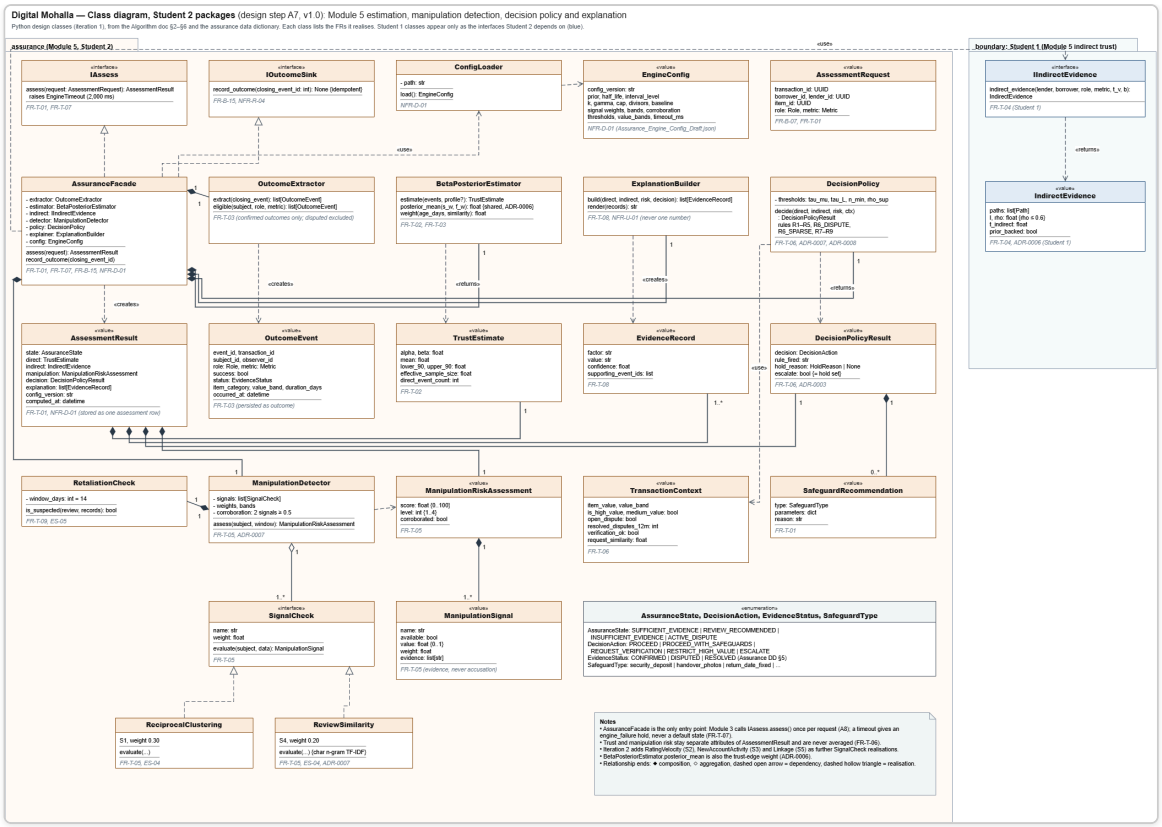


Figure E.2: Domain Class Diagram: Student 2 (Bayesian Engine & Estimation)

Class model for Student 3 (Minahil Kashif) detailing application services, moderation workflows, and Expo UI components:

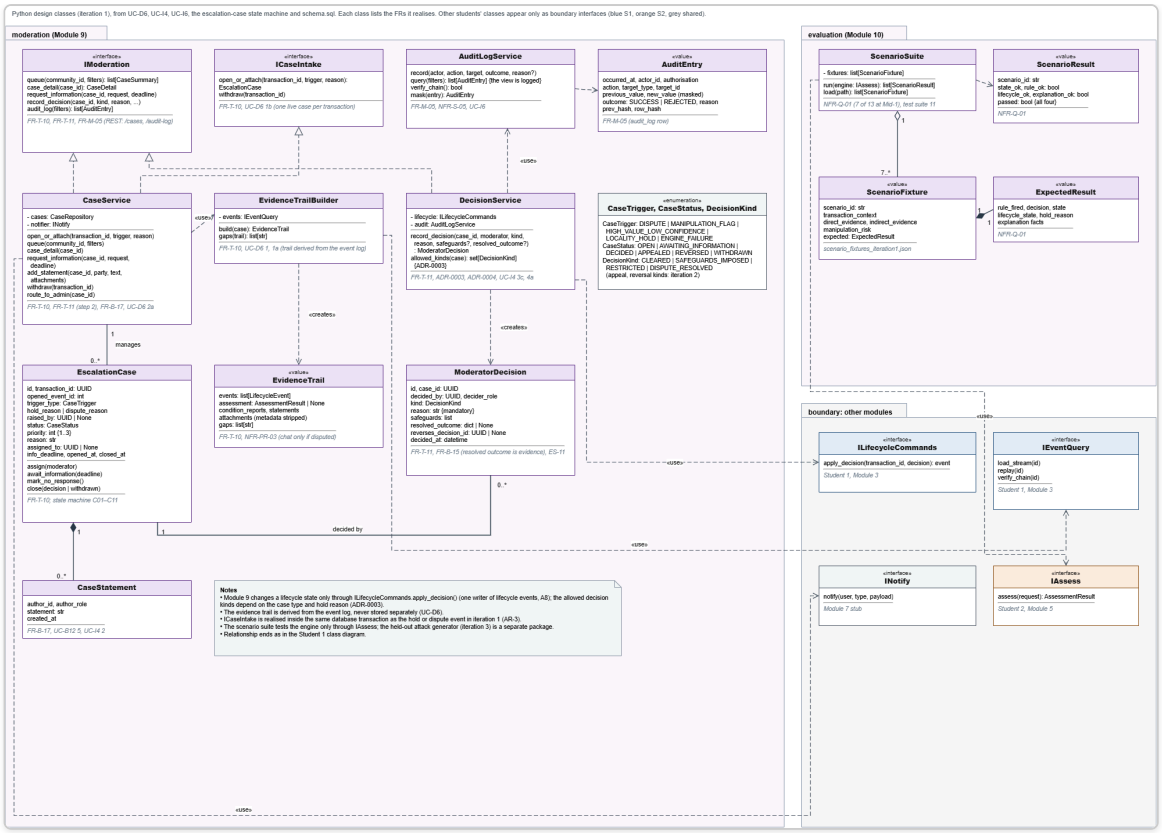


Figure E.3: Domain Class Diagram: Student 3 (Application & Presentation Client)

E.2 User Interface Wireframes and Workflows

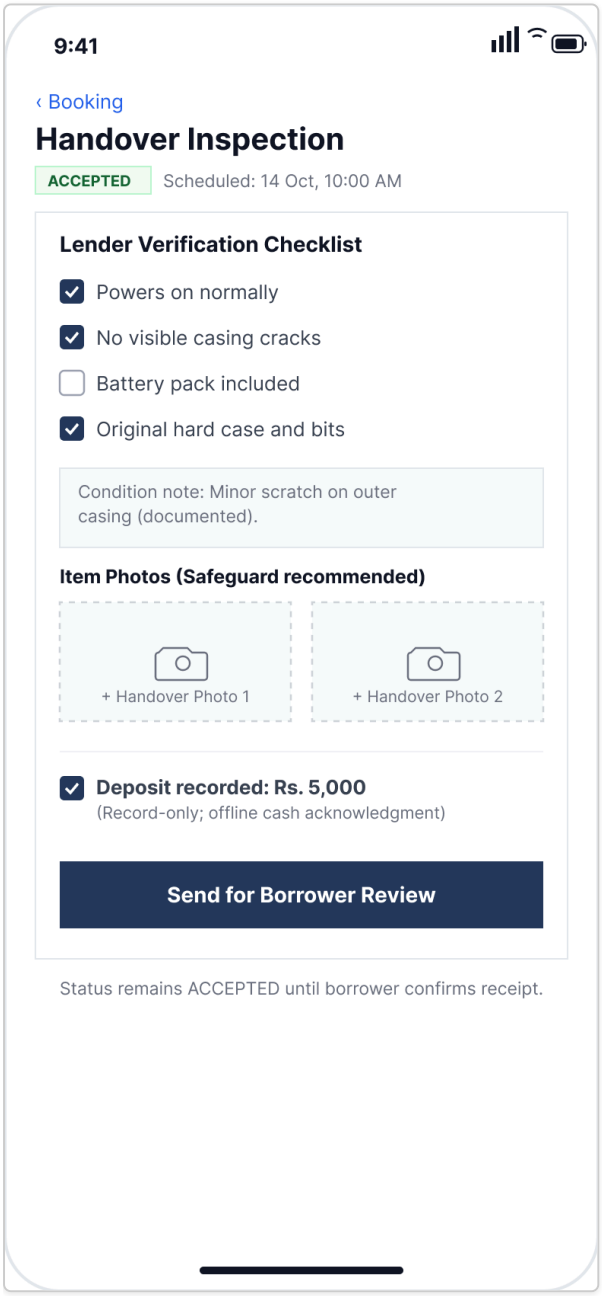


Figure E.4: Lender Handover Inspection Checklist (W3a)

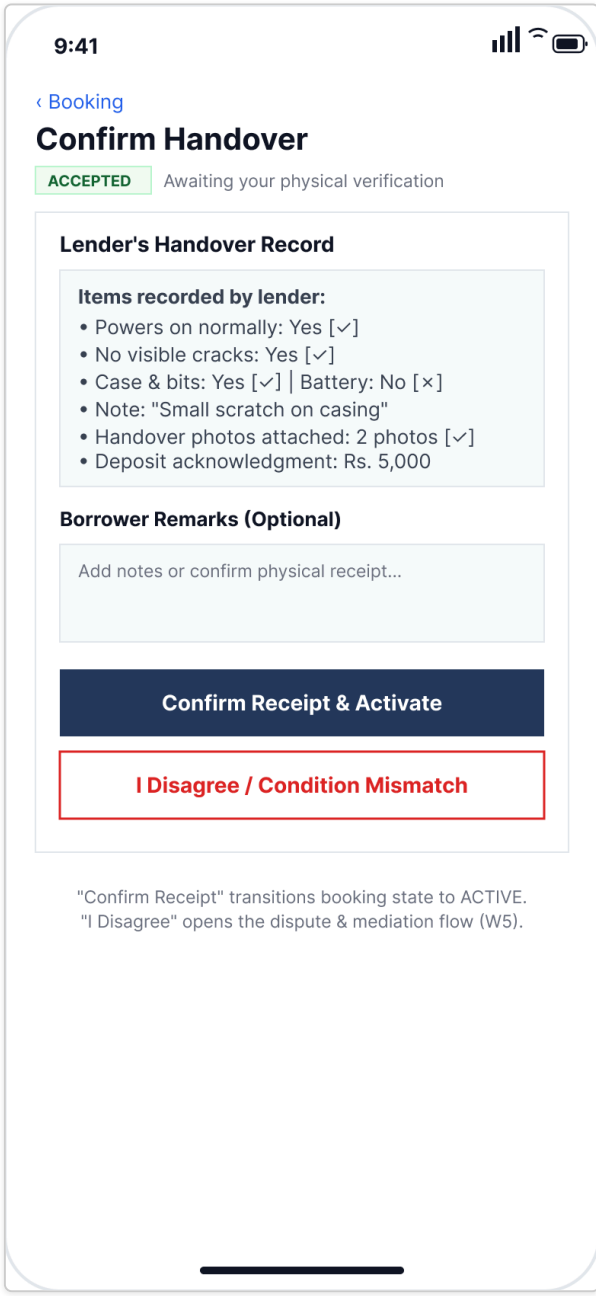


Figure E.5: Borrower Handover Confirmation (W3b)

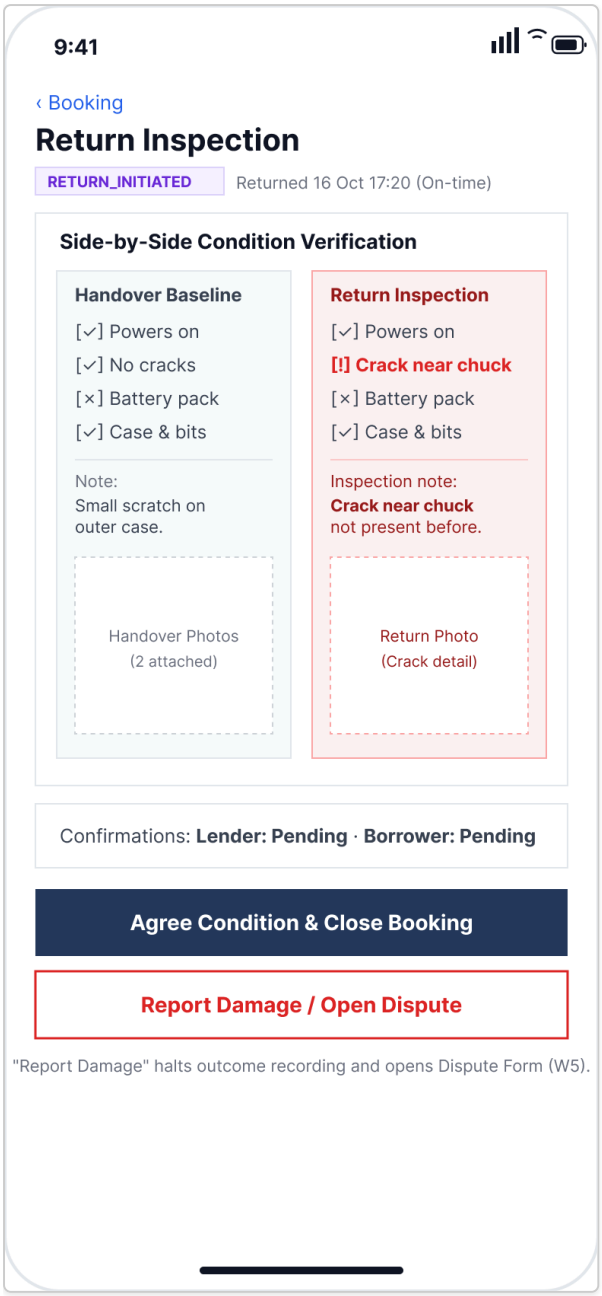


Figure E.6: Return Condition Photo Comparison (W4)

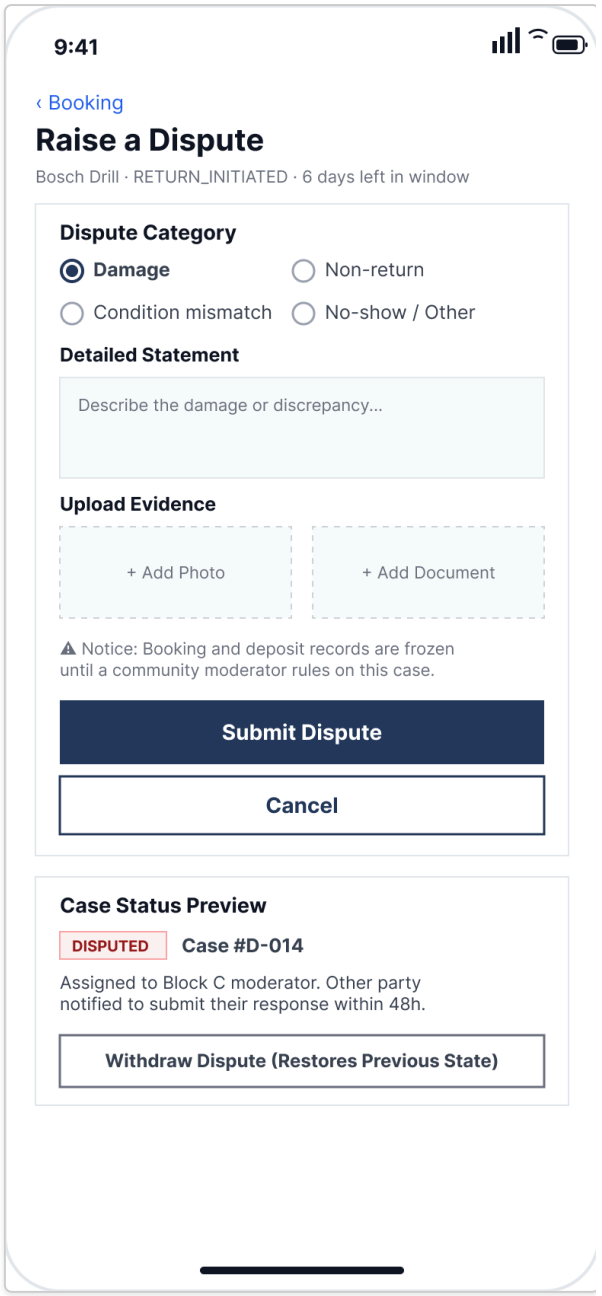


Figure E.7: Contested Return Dispute Form (W5)

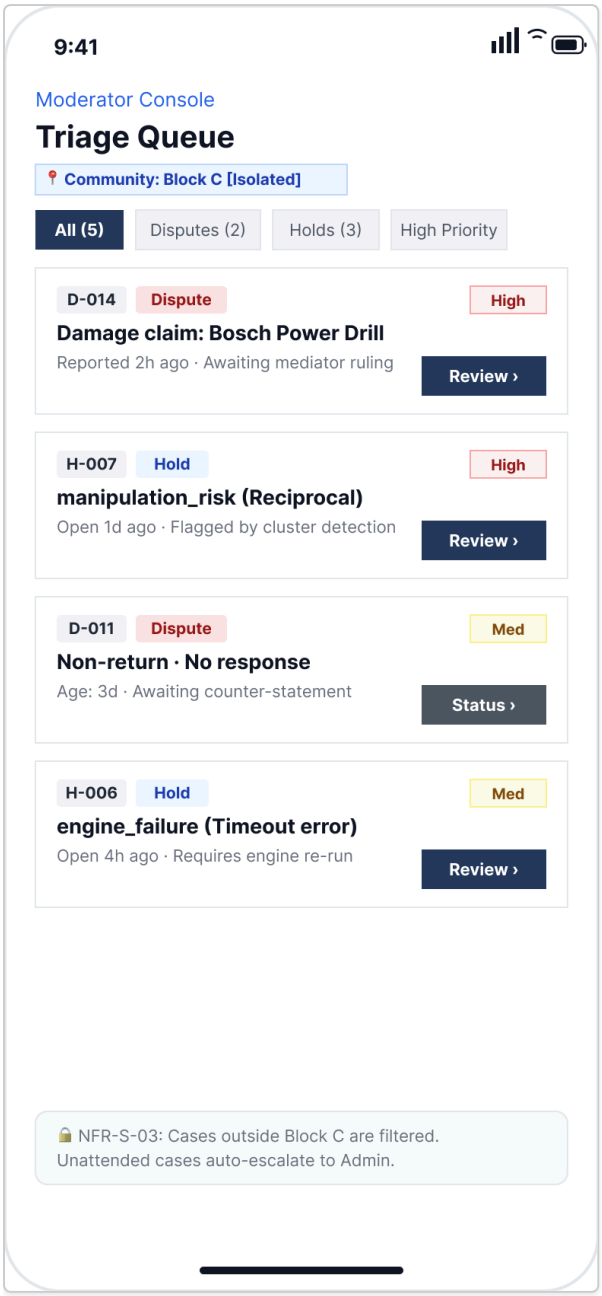


Figure E.8: Community Moderator Escalation Queue (W6)

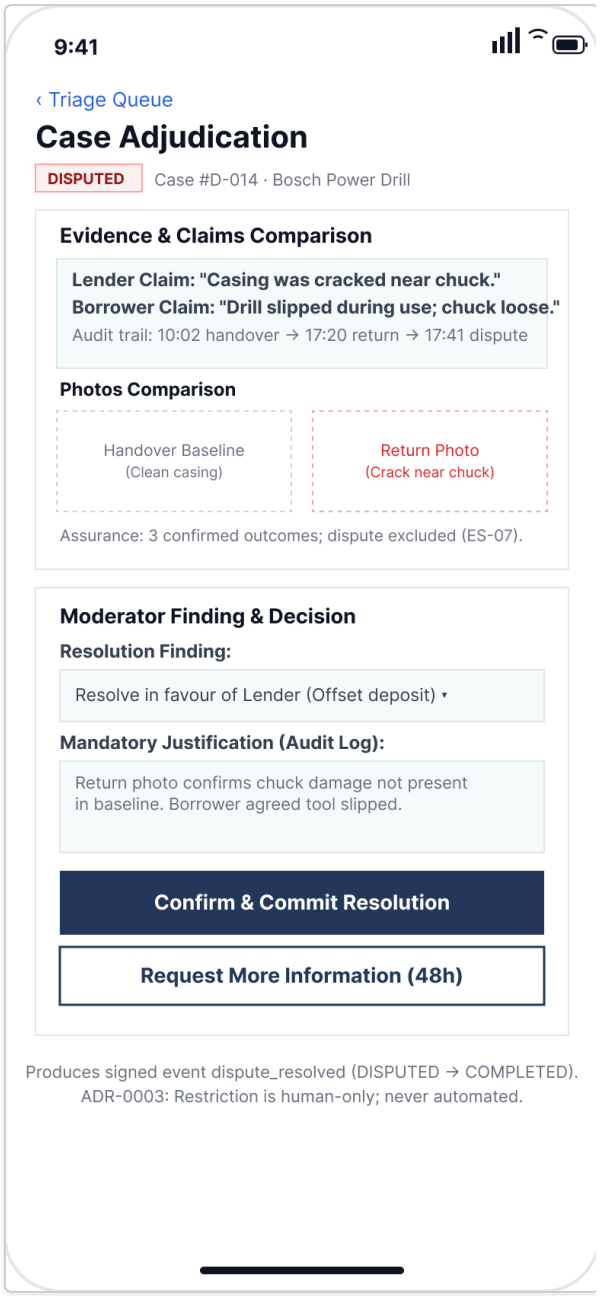


Figure E.9: Moderator Case Adjudication and Ruling Screen (W7)

Appendix F: Meeting and Progress Evidence

Table F-1: Supervisory Coaching and Team Engineering Decision Log

Date	Meeting Type / Members	Key Decisions / Directives Resolved	Evidence
2026-09-18	Proposal Defense / All	Addressed panel directives on technical depth, Bayesian trust, and privacy.	Form 10 Register
2026-09-26	Supervisory Checkpoint / Dr. Ali	Approved move toward Bayesian trust modeling and event-sourced lifecycle.	Meeting Log #1
2026-10-04	Supervisory Checkpoint / Dr. Ali	Verified UML models; directed isolation of assurance behind `IAssess` boundary.	Meeting Log #2
2026-10-11	Team Integration Meeting / All	Decisions D10–D19: Approved Part A integrated baseline (12/12 checks passed).	Integration Report
2026-10-13	Team Sprint Meeting / All	Decisions D10–D17: Approved ADR-0006 experimental edge weighting, similarity D17.	Team Decisions Log

References

1. A. Jøsang, R. Ismail, and C. Boyd, 'A survey of trust and reputation systems for online service provision,' *Decision Support Systems*, vol. 43, no. 2, pp. 618–644, 2007.
2. S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, 'The EigenTrust algorithm for reputation management in P2P networks,' in *Proc. 12th Int. Conf. World Wide Web (WWW)*, 2003, pp. 640–651.
3. P. Massa and P. Avesani, 'Trust-aware recommender systems,' in *Proc. 2007 ACM Conf. Recommender Systems (RecSys)*, 2007, pp. 17–24.
4. M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley Professional, 2002.
5. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
6. S. Kumar, F. Spezzano, V. S. Subrahmanian, and C. Faloutsos, 'Edge weight prediction in weighted signed networks,' in *IEEE 16th Int. Conf. Data Mining (ICDM)*, 2016, pp. 221–230.
7. P. Resnick, R. Zeckhauser, E. Friedman, and K. Kuwabara, 'Reputation systems,' *Communications of the ACM*, vol. 43, no. 12, pp. 45–48, 2000.
8. S. Ramírez, 'FastAPI: High performance web framework for building APIs with Python,' 2024. [Online]. Available: https://fastapi.tiangolo.com
9. PostgreSQL Global Development Group, 'PostgreSQL 16 Documentation: Exclusion Constraints and btree_gist,' 2024. [Online]. Available: https://www.postgresql.org/docs/16/
10. Expo / 650 Industries, 'Expo Router and React Native Framework Documentation,' 2024. [Online]. Available: https://docs.expo.dev